PoisonSpot: Precise Spotting of Clean-Label Backdoors via Fine-Grained Training Provenance Tracking

Philemon Hailemariam
philemon@umich.edu
University of Michigan, Dearborn
Dearborn, MI, USA

Birhanu Eshete
birhanu@umich.edu
University of Michigan, Dearborn
Dearborn, MI, USA

Abstract

Relying on untrusted data exposes machine learning models to backdoor attacks, where adversaries poison training data to embed hidden behaviors. Existing defenses struggle against increasingly stealthy attacks, particularly clean-label backdoor attacks, due to their inability to monitor fine-grained impact of individual training samples on model updates.

In this paper, we present PoisonSpot, a novel system that precisely detects clean-label backdoor attacks by using *fine-grained training provenance tracking*, inspired by dynamic taint tracking. PoisonSpot captures and analyzes the impact of individual training samples on model parameter updates throughout the training process. By attributing poisoning scores to suspect samples based on their impact lineage, PoisonSpot allows for accurate identification and rejection of samples carrying backdoor triggers.

We evaluate PoisonSpot on multiple benchmark datasets and attack scenarios, demonstrating its superior performance compared to the state-of-the-art clean-label backdoor poisoning defense. PoisonSpot consistently achieves high true positive rates, low false positive rates, and effectively mitigates backdoor attacks, even under adaptive adversarial strategies. Furthermore, PoisonSpot operates efficiently in various training settings, including retraining and fine-tuning regimes, demonstrating its robustness and scalability.

CCS Concepts

• **Security and privacy**; • **Computing methodologies** → **Artificial intelligence**;

Keywords

Training Provenance, Clean-Label Backdoors, Backdoor Defenses

ACM Reference Format:
Philemon Hailemariam and Birhanu Eshete. 2025. PoisonSpot: Precise Spotting of Clean-Label Backdoors via Fine-Grained Training Provenance Tracking. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), October 13–17, 2025, Taipei, Taiwan.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3719027.3744802

1 Introduction

Machine learning (ML) plays a key role in critical applications, from malware detection and autonomous vehicles to facial recognition and financial forecasting. To ensure ML models are consistently able to produce correct and reliable output, they need to be retrained as new input samples become available. This leads ML model trainers to retrain their model with data collected through various (potentially untrusted) sources. This opens up the opportunity for *backdoor data poisoning attacks* [1, 5, 16, 24, 35, 36, 42, 45], where adversaries manipulate a portion (typically a small percentage) of training data in order to have the ML model misbehave in favor of the adversarial goals. Backdoor attacks have become stealthier and harder to detect with prior defenses making existing detection approaches [7, 28, 41] insufficient.

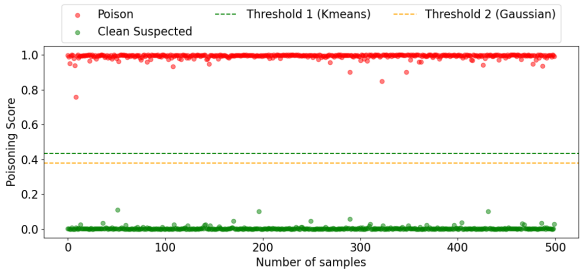


Figure 1: PoisonSpot typical detection result against a state-of-the-art clean-label backdoor attack [46] at a low poison ratio (1% of training set) on the CIFAR-10 dataset.

In pursuit of exploring new frontiers for precise backdoor detection, in this paper, we ask the following question: *when retraining a model on a combination of clean and newly-acquired (potentially poisonous) samples, can we instrument the training session, track the impact of each new sample on model parameter updates, and leverage the impact to attribute a poison score to suspect samples, so as to sanitize training data from backdoor trigger-carrying samples?*

Prior work: In the context of this question, prior backdoor poisoning detection approaches have explored the notion of data provenance. In particular, provenance-based defenses have relied on metadata [3], data source trustworthiness [4], or crytographically authenticated source metadata [38] to analyze metadata and source trustworthiness and decide whether to remove suspicious samples from a training set. These approaches face two main challenges. Firstly, both metadata and data source trustworthiness could be compromised or faked by a seemingly honest data source. This is primarily because the model trainer does not have control over the provenance data, which leaves the adversary with room for adaptive manipulation of the provenance data. Secondly, recent backdoor attacks (e.g., [46]) employ stealthy backdoor triggers that are hard to uncover in the absence of high-resolution visibility into the impact of individual training samples on model parameter

This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '25, Taipei, Taiwan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1525-9/2025/10
https://doi.org/10.1145/3719027.3744802

updates. These two challenges motivate us to design a solution that ensures that the model trainer is able to capture and analyze the impact of each sample in newly acquired (potentially poisonous) dataset to determine its contribution to model parameter updates (and hence model inference results).

Approach Overview: In this paper, we introduce PoisonSpot, a practical system that precisely detects clean-label backdoor attacks through *fine-grained training provenance tracking* inspired by the key concepts in dynamic taint tracking (DTT). In particular, PoisonSpot monitors the impact of newly acquired data samples on model updates throughout the training process. PoisonSpot captures training provenance at both batch and sample levels and analyzes these data to learn a model that assigns poisoning scores to suspect samples. PoisonSpot works well in practice because of the following key aspects: (1) it tags each new sample as suspect upon arrival (this ensures that tracking is deterministic despite the stochastic nature of the training regime); (2) it tracks suspect samples at the right vantage point by capturing impact/lineage at the model parameter update so that any poisoning signal, however subtle, leaves traces of its impact during model parameters update at the batch-level; and (3) it tracks training provenance at the right level of granularity (batch-level tracking provides the highest resolution of visibility on impact of poisonous samples on model parameters).

Evaluation Overview: We evaluate PoisonSpot across multiple benchmark datasets for image classifiers. It consistently outperforms current state-of-the-art defense ([31]) in detecting backdoor attacks across datasets, attacks, poisoning ratios, and training regimes. Figure 1 shows a sample for a typical detection result of PoisonSpot. It offers strong performance in upstream detection metrics (high true positive rates and low false positive rates) and in downstream metrics (maintaining low attack success rates and high model accuracy post-defense). PoisonSpot remains resilient in the face of adaptive attacks that attempt to bypass detection by manipulating poison ratios, using stronger perturbations, or a mixture of attacks. PoisonSpot's robustness extends to different training settings, whether retraining from scratch or fine-tuning, demonstrating its flexibility and adaptability. To our knowledge, PoisonSpot is the first work to effectively tailor the key ingredients of dynamic taint tracking for fine-grained training provenance tracking to precisely detect clean-label backdoors.

Contributions: This paper makes the following contributions to the detection of clean-label backdoor poisoning:

- We tailor dynamic taint tracking in the context of clean-label backdoor poisoning detection and design a precise backdoor detection system, PoisonSpot, that leverages the provenance (lineage) of impact of suspicious samples on model parameter updates during training. In doing so, PoisonSpot enables a model trainer to track suspicious training samples at the granularity of each data-point and model parameter update iteration.
- We empirically demonstrate the high recall and negligible false positive rate of PoisonSpot across multiple datasets, attacks, poison ratios, and training settings.
- We demonstrate PoisonSpot's resilience against three classes of adaptive attacks that exploit poison ratio, perturbation strength, and attack diversity.
- We make a reproducible PoisonSpot code available at https://doi.org/10.5281/zenodo.15660315.

2 Background

We begin with a brief background on supervised ML preliminaries, backdoor poisoning attacks, and dynamic taint tracking.

2.1 Supervised ML Preliminaries

Training. We focus on supervised ML models. Given a set of labeled training samples $X_{train} = (X_i, y_i) : i \leq n$, where X_i is a training example and y_i is the corresponding label, the objective of training a ML model f_θ, parameterized by a weight vector θ, is to minimize the expected loss over all (X_i, y_i) computed as: $J(\theta) = \frac{1}{n}\sum_1^n l(\theta, X_i, y_i)$. In ML models such as deep neural networks (DNNs), the loss minimization problem is typically solved using stochastic gradient descent (SGD) by iteratively updating θ as: $\theta = \theta - \epsilon \cdot \Delta_\theta \sum_{i=1}^n l(\theta, X_i, y_i)$, where Δ_θ is the gradient of the loss with respect to the weights θ; X is a randomly selected set (e.g., a *mini-batch*) of training examples drawn from X_{train}; and ϵ is the *learning rate* that controls the magnitude of change in θ.

Testing: Let X be a d-dimensional feature space and Y be a k-dimensional output space, with underlying probability distribution $Pr(X, Y)$, where X and Y are random variables for the feature vectors and the labels, respectively. The objective of testing a ML model is to perform the mapping $f_\theta : X \rightarrow Y$. The output of f_θ is a k-dimensional vector and each dimension represents the probability of input belonging to the corresponding class/label.

2.2 Backdoor Poisoning Attacks

ML is vulnerable to adversarial manipulation during training time via backdoor poisoning attacks [1, 5, 16, 24, 29, 35, 36, 42, 45]). Adversaries manipulate a subset of training data to influence model behavior (e.g., targeted misclassification of samples). Backdoor attacks are broadly divided into two categories: dirty-label and clean-label attacks. Dirty-label backdoor attacks alter both the label and features of a sample. Initially, these attacks used sample-independent, visible patterns as triggers[16], but have since evolved into more covert forms, using sample-specific [22] or visually imperceptible triggers [8, 25, 30, 48]. On the other hand, clean-label backdoor attacks maintain that the manipulated features remain semantically aligned with the correct labels. Attacks in this category vary from using adversarial perturbation-based [40] to feature-collision-based [34] and optimization-based attacks [15, 37, 46]. Our focus for this paper is on clean-label backdoor attacks.

2.3 Dynamic Taint Tracking

Dynamic taint tracking (DTT) for vulnerability identification tracks the flow of untrusted data through a program as it executes [9]. It bootstraps tracking by marking (tainting) data that come from (untrusted) *sources* such as user inputs. As the program executes, DTT monitors the tainted data on how it propagates through the program to check if it affects sensitive functions, called *sinks* (e.g., database query functions and file operations) [9, 32]. DTT typically consists of three steps: (1) taint source identification and tagging, (2) taint propagation and analysis, and (3) policy enforcement. In (1) the goal is to determine which data/inputs to the program are from untrusted sources and should be considered tainted and then assigning a taint mark to such data. This is often done by instrumenting the program's execution. In (2) the goal is to define rules

that specify how taint should propagate through the program's operations. For instance, if a tainted input is used in a calculation of a new result, the result needs to be tainted as well. The focus of the analysis is to ensure that no tainted data is used in a sensitive context (e.g., a database query function) without proper validation and/or sanitization. In (3) the goal is to apply policies that define what measures to take when tainted data reach sensitive sinks. Measures could include terminating program execution, alerting a user/administrator, sanitizing data, or logging the event for later analysis. The key intuition behind the design of PoisonSpot is based on DTT, the details of which are in §5 and §6.

3 Related Work

Previous work has detected backdoor by examining their differences from clean ones in the embedding space, using methods such as Singular Value Decomposition [18], Gram matrix [27], K-Nearest-Neighbors [20], and feature decomposition [39]. Other methods utilize intermediate neural activation [7], gradients [6], resistance to augmentations [13], high-frequency artifacts [47], and early-stage training loss [23]. We position PoisonSpot with respect to previous provenance-based and non-provenance-based defenses.

Provenance-Based Defenses. Chen et al.[7] propose an *activation clustering*-based method to detect poisonous training samples based neuron activations from the last hidden layer, segmented by label, and clustered into two activation groups using K-means, with the poisonous cluster identified through *exclusionary reclassification*, *relative size comparison*, and *Silhouette Score*. Baracaldo et al. [3] leverage data provenance for pre-training data sanitization. Each data point is linked with its "provenance record" or meta-data, and untrusted data points are grouped according to *provenance signatures*. The model is trained with and without each group, and the filtered model's performance is compared to the unfiltered model. If the filtered model outperforms the unfiltered one, the group is identified as poisonous and removed. The approach outperforms *Reject On Negative Impact* (RONI) [28], which requires the model to be trained once per data-point, whereas [3] requires retraining on only a fraction of untrusted data-points. Baracaldo et al. [4] extended [3] by considering collusion attacks and assuming that the data sources are independent of each other. Jackson [19] explored training metrics (e.g., loss, accuracy) as clues for poisoning.

Non-Provenance-Based Defenses. The state-of-the-art backdoor defenses are all non-provenance-based. They are classified based on how the defense is implemented to protect against backdoor attacks[10]. They include sanitization of training data [14, 18, 27, 31, 33], robust training [23, 44], model inspection [7], model sanitization [43], trigger reconstruction [41], and sanitization of test data [13]. Pan and Zhang [31] proposed the latest backdoor detection approach named ASSET, which induces opposite model behaviors between poisoned and clean samples. This is achieved through two simultaneous optimizations: one minimizes a certain loss function on the clean base set, while the other maximizes the same loss on the entire poisoned training dataset, including both clean and poisoned samples. The second optimization's effect on clean samples is offset by the first, leaving their loss largely unchanged, whereas the poisoned samples, affected only by the second optimization, have maximized loss, making the two separable through differing loss values. However, in cases of low poison ratios, the model can prioritize optimizing for clean samples, limiting its performance, a challenge that was mitigated by incorporating an inner loop to concentrate on poison samples. ASSET outperformed previous defenses (SPECTRE [18], AC[7], Beatrix [27], CT[33], ABL[23], and STRIP[13]). It is also the first defense to successfully defend against the state-of-the-art clean label attack (Narcissus[46]), and is the current state-of-the-art clean-label backdoor detection method. Hence, in §7.3 we compare PoisonSpot with ASSET.

Compared to non-provenance-based defenses, PoisonSpot has several advantages. First, PoisonSpot is more resistant to feature collision attacks, where adversaries craft poisoned samples whose feature representations are close to clean samples [34, 40]. Second, PoisonSpot is more resistant to gradient alignment attacks in which adversaries manipulate poisoned samples to align gradients with those of clean samples [37, 46]. Since it relies on weight updates rather than on data or feature space distribution, PoisonSpot is also robust to adaptive attacks. Matching clean and poisonous weight updates is hard since the adversary has not control over how the model learns the backdoor trigger. If a sample is poisonous, some of the weight parameters will be updated distinctively compared to clean samples:PoisonSpot leverages this difference by capturing the provenance data to detect the poisonous samples.

ASSET [31] is the current state-of-the-art detection technique, which is data dependent and thus is vulnerable to an adaptive attack where the adversary adds noise to the samples to evade the detection if the adversary knows the parameters of ASSET's detector (white-box scenario). Although the authors fix this through unlearning, it may often be at the cost of a severe drop in clean model accuracy. There is a case where they reported over 20% drop in clean model accuracy for Label-Consistent attack [40] in their paper[31]. PoisonSpot would not be vulnerable to such an issue since the adversary would not have control over the captured provenance data. Table 1 shows how PoisonSpot compares with our benchmark defense, ASSET.

Criteria	ASSET [31]	PoisonSpot
Resilient to feature collision attacks	✓	✓
Resilient to gradient alignment attacks	✓	✓
Resilient to white-box adaptive attacks	✗(may come at a cost of low model ACC)	✓

Table 1: Comparison of PoisonSpot and ASSET [31].

4 Threat Model and Problem Definition

4.1 Threat Model

We consider two principal parties: a model trainer $\mathbb{T}$ and a backdoor poisoning adversary $\mathbb{A}$, with the following assumptions about their goals, knowledge, and capabilities:

Model Trainer $\mathbb{T}$ maintains a clean dataset $\mathcal{D}_c$ and trains a model $\mathcal{F}$ (e.g., a spam detector). After initial deployment, $\mathbb{T}$ periodically augments $\mathcal{D}_c$ with newly acquired, untrusted samples $\mathcal{D}_u$ and then (a) retrains $\mathcal{F}$ from scratch on $\mathcal{D}_c \cup \mathcal{D}_u$ or (b) fine-tunes the existing model on $\mathcal{D}_u$ alone. Before updating $\mathcal{F}$, $\mathbb{T}$ aims to identify and remove poisoned examples in $\mathcal{D}_u$. We assume that $\mathbb{T}$:

• is able to determine which fraction of $\mathcal{D}_u$ (with respect to $\mathcal{D}_c$) to use to retrain $\mathcal{F}$;

• decides how often to retrain $\mathcal{F}$ depending on whether there is a need to do so (e.g., enough new training samples likely to improve model accuracy up on retraining);
• possesses a trusted computing base (e.g., GPUs, ML frameworks) whose integrity is guaranteed between training sessions;
• archives all model artifacts and logs in secure storage, enabling forensic recovery if needed.

Backdoor Poisoning Adversary $\mathbb{A}$ seeks to covertly inject trigger-embedded samples into $\mathcal{D}_u$ so that, once retrained, $\mathcal{F}$ classifies any input stamped with the trigger as a target label chosen by $\mathbb{A}$, all while maintaining nominal accuracy on benign inputs. Conforming to the clean-label backdoor setting [1, 27, 31, 34, 37, 40, 46], we assume that $\mathbb{A}$:
• can manipulate only the features (not the labels) of a subset of new training points in $\mathcal{D}_u$;
• may adapt its trigger designs and poisoning strategies in response to $\mathbb{T}$'s defenses;
• cannot tamper with $\mathbb{T}$'s secure training infrastructure or access the clean data $\mathcal{D}_c$.

4.2 Problem and Challenges

Model Trainer's Dilemma. When samples $\mathcal{D}_u$ arrive, $\mathbb{T}$ faces a fundamental dilemma on the veracity of $\mathcal{D}_u$. On the one hand, $\mathbb{T}$ is tempted to augment $\mathcal{D}_u$ with $\mathcal{D}_c$ and retrain $\mathcal{F}$ on $\mathcal{D}_c \cup \mathcal{D}_u$ or finetune $\mathcal{F}$ on $\mathcal{D}_u$. On the other hand, $\mathbb{T}$ is uncertain whether $\mathcal{D}_u$ contains samples crafted by $\mathbb{A}$ to poison $\mathcal{D}_c$ and subsequently activate the backdoor behavior learned by $\mathcal{F}$ in favor of $\mathbb{A}$'s goals.

Goals. Given a clean dataset $\mathcal{D}_c$ and let $\mathcal{D}_u$ be disjoint with $\mathcal{D}_c$ and is a small fraction (e.g., 3%-5%) of $\mathcal{D}_c$. Our goal is to instrument the training process so as to capture and characterize how $\mathcal{D}_u$ impacts the model parameters when retraining $\mathcal{F}$ on $\mathcal{D}_c \cup \mathcal{D}_u$ or fine-tuning it on $\mathcal{D}_u$. More precisely, our goal is to track the impact of each sample in $\mathcal{D}_u$ throughout the training trajectory to find clues indicative of whether or not a model re-trained on $\mathcal{D}_c \cup \mathcal{D}_u$ or fine-tuned on $\mathcal{D}_u$ is poisoned by $\mathcal{D}_u$. In doing so, our objective is to enable $\mathbb{T}$ to eliminate poisonous samples in $\mathcal{D}_u$ and retrain $\mathcal{F}$ on $\mathcal{D}_c$ enhanced with non-poisonous samples (if any) in $\mathcal{D}_u$.

Challenges. A natural challenge to tackle here stems from the innate non-determinism of reshuffling before batch sampling over multiple epochs of model training. In addition, other contributors are factors such as mini-batch size choices, diversity of backdoor triggers, training settings (e.g., from scratch or fine-tuning), and percentage of poison data with respect to training data size.

4.3 Design Requirements

To precisely spot and remove clean-label backdoor samples from a training set, PoisonSpot should meet the following requirements:
• **High True Positive Rate (TPR).** Obtaining a high TPR is crucial for identifying all poisonous samples in $\mathcal{D}_u$ to sanitize the training data from the poisonous data.
• **Low False Positive Rate (FPR).** Although all samples in $\mathcal{D}_u$ are labeled suspect until they are proven clean, it is in the best interest of $\mathbb{A}$ to mix clean and poison samples to confuse PoisonSpot. In this context, our objective is to minimize the number of clean samples reported as poisonous to use clean samples in $\mathcal{D}_u$ towards potential improvement in $\mathcal{F}$'s accuracy up on retraining.
• **Cross-Attack Robustness.** PoisonSpot needs to detect clean-label backdoors with high TPR at low FPR across a family of attacks that may employ various backdoor trigger injection strategies.
• **Adaptive Resilience.** PoisonSpot needs to be resilient in the face of adaptive attack strategies mounted by $\mathbb{A}$.

5 PoisonSpot Design Intuition and Assumptions

Before we describe the technical details of PoisonSpot, we provide a high-level overview and the assumptions that guide its design.

5.1 Overview

At the heart of the design of PoisonSpot is *training provenance tracking (TPT)*, a novel application of dynamic taint tracking (DTT) principles, introduced in §2.3, to the problem of clean-label backdoor detection. TPT continuously monitors the influence of every new, potentially tainted sample, drawn from the untrusted set $\mathcal{D}_u$, as it flows through the training pipeline. By treating each sample in $\mathcal{D}_u$ as *tainted* after ingestion, TPT records how these data propagate through successive loss calculations, gradient calculations, and, crucially, the *model parameter update* function, which we designate as the primary *sink*. We focus on the parameter update step for two reasons. First, it is the *final impact point* where computed gradients are applied, so any downstream effect of poisoned samples must manifest here. Second, parameter updates *subsume earlier operations*, the losses and gradients that those losses generate, making them the most comprehensive locus for attribution. Whenever a training batch includes poisoned samples, TPT flags the entire batch as tainted/suspect and quantifies the contribution of each sample to the subsequent update of the parameters.

Over the course of multiple epochs, TPT accumulates a detailed lineage, the *training provenance*, for every sample in $\mathcal{D}_u$ (and, by extension, for clean samples in $\mathcal{D}_c$ when used as references). We then distill this provenance into a *probabilistic poisoning score*: a measure of how distinguishable the perturbation induced by a sample's update is from that of clean data. Because batch reshuffling ensures that poisoned examples reappear throughout training, their aggregated influence on parameter updates becomes statistically separable from normal variation. Our probabilistic scoring method dynamically adapts to various poisoning behaviors, making PoisonSpot *attack agnostic*.

Once all epochs are complete, PoisonSpot applies an empirically chosen threshold to these scores, partitioning $\mathcal{D}_u$ into likely clean and likely poisoned subsets. By excluding high-scoring (poisonous) samples and retraining on sanitized data, PoisonSpot restores confidence in the updated model without sacrificing the benefits of newly acquired non-poisonous examples.

Importantly, PoisonSpot is *modality-agnostic*: whether it operates on images, text, or speech, the method of tagging inputs and auditing their impact on parameter updates remains the same. The overall pipeline (illustrated in Figure 2) unfolds in three phases: Phase I: *Training Provenance Capture*: focuses on tagging and per-sample model parameter update tracing during training; Phase II: *Training Provenance Analysis*: aggregates model parameter update distances into per-sample impact profiles; and Phase III: *Poisoning Score Aggregation & Thresholding*: computes poisoning scores and filters out poisonous samples.

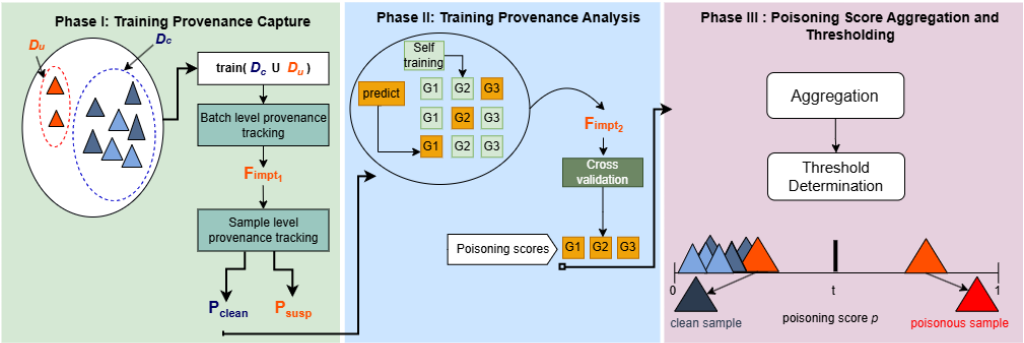


Figure 2: Overview of PoisonSpot. In Phase I, $\mathcal{D}_c$ refers to vetted clean samples; $\mathcal{D}_u$ refers to suspect samples acquired from potentially untrusted sources; $\mathcal{P}_c$ is provenance data of clean samples; $\mathcal{P}_s$ refers to provenance data of suspect samples captured based on their impact on model parameter updates during training; and $\mathcal{F}_{impt1}$ is provenance feature vector (model parameter dimensions) selected based on an empirically established threshold over the difference of batch-level training provenance data. In Phase II, $\mathcal{F}_{impt2}$ refers to provenance feature vector of important provenance features (model parameter dimensions) selected based on an empirically established threshold over observations of sample-level training provenance data. In Phase III, t refers to an empirically established poison score threshold while p is probabilistic poison score in the range [0,1].

5.2 Assumptions

We make the following assumptions that underpin the key design choices in PoisonSpot. We will refer to them throughout our design description and present evidence of their validity in §7.2.

A1: Poisonous samples affect the model parameter updates significantly differently than clean samples.

A2: A consistent set of parameters is uniquely affected by poisonous samples.

A3: Individual sample impact can be isolated via single-sample training, closely reflecting real-time batch impact.

A4: Cross-validation models effectively differentiate between clean and poisonous sample impacts based on provenance data.

A5: When at least 5% of the suspect set ($\mathcal{D}_u$) is poisoned, the statistical distinction in update signatures becomes reliably detectable.

6 PoisonSpot Design Details

We now present the technical details of PoisonSpot.

6.1 Training Provenance Capture

Training provenance capture (Phase I in Figure 2) is conducted in two distinct stages to effectively and efficiently identify and analyze the impacts of suspect samples on model parameter updates. The first stage is *batch-level training provenance capture*, where training is performed over a few initial epochs on top of an accurate poisoned model[1] to identify provenance features most relevant to poisoning. The second stage is the *capture of sample-level provenance* aimed at enhancing the provenance analysis by focusing on individual samples to determine their specific contribution to the overall poisoning effect. The provenance data from this stage are utilized for self-training, during which samples are scored based on their "poisonousness" (referred to as poisoning scores). For training from scratch, sample-level provenance is captured during additional epochs beyond initial training. For fine-tuning, all fine-tuning epochs are leveraged for provenance capture.

By restricting provenance capture to a few epochs rather than the entire training process, PoisonSpot significantly reduces compute time overhead and memory usage without compromising detection effectiveness. This efficiency comes from the observation that the effects of poisoning persist in later training stages, once the model reaches satisfactory accuracy. For larger datasets, additional efficiency gains can be achieved through GPU parallelization and by narrowing the focus to critical model parameters, thereby reducing the dimensionality of the captured provenance.

6.1.1 Batch-Level Training Provenance Capture

Batch-based sampling model parameter update is an established regime for training DNNs. Aligned with this best practice, PoisonSpot first captures batch-level training provenance to identify features most relevant to poisoning. The process begins by training a model $\mathcal{F}$ on $\mathcal{D}_c \cup \mathcal{D}_u$ for a sufficient number of epochs to achieve satisfactory accuracy. Once this baseline training is completed, training continues for a few additional epochs to specifically capture provenance data. These additional epochs are focused on monitoring the training trajectory and identifying batch-level provenance features that influence model parameter updates.

To isolate the most relevant features, PoisonSpot performs training provenance feature selection by analyzing the features that significantly impact model parameter updates for batches containing poisonous samples, compared to those containing only clean samples. This process is repeated for each epoch, and the intersection of the identified features across epochs is retained. This ensures that only features with consistent influence throughout training are considered. The detailed steps for batch-level provenance capture (Algorithm 1) include: tainted batch identification, creating matched batch variations, parameter updates and provenance capture, and training provenance pruning.

[1] trained on $\mathcal{D}_c \cup \mathcal{D}_u$ and has reached acceptable accuracy despite being backdoored

Algorithm 1: Batch-Level Training Provenance Capture

Input: $\mathcal{D}_c$, $\mathcal{F}$, $\mathcal{D}_u$
Output: $\mathcal{F}_{impt1}$
1 **for** *each epoch* $\mathcal{E}$ **do**
2 **for** *each batch* $\mathcal{B}$ **do**
3 $\mathcal{B}_s \leftarrow \mathcal{B} \cap \mathcal{D}_u$;
4 **if** $\mathcal{B}_s \neq \emptyset$ **then**
5 $\mathcal{B}_{c1} \leftarrow$ `sample-from`($\mathcal{B}$); `// or sample-from(`$\mathcal{D}_c$`)`
6 $\mathcal{B}_{c2} \leftarrow$ `sample-from`($\mathcal{B}$); `// or sample-from(`$\mathcal{D}_c$`)`
7 **for** *each class in* $\mathcal{B}_s$ **do**
8 $\mathcal{U}_1 \leftarrow$ `parameter-update`($\mathcal{F}_{\mathcal{B}}, \mathcal{B}_{c1}[class]$);
9 $\mathcal{U}_2 \leftarrow$ `parameter-update`($\mathcal{F}_{\mathcal{B}}, \mathcal{B}_{c2}[class]$);
10 $\mathcal{U}_3 \leftarrow$ `parameter-update`($\mathcal{F}_{\mathcal{B}}, \mathcal{B}_{s}[class]$);
11 $\mathcal{P}_{susp_{class}}[\mathcal{E}, \mathcal{B}] \leftarrow$ `distance`($\mathcal{U}_2, \mathcal{U}_3$);
12 $\mathcal{P}_{clean_{class}}[\mathcal{E}, \mathcal{B}] \leftarrow$ `distance`($\mathcal{U}_1, \mathcal{U}_2$);
13 **end**
14 **end**
15 **end**
16 **end**
17 $\mathcal{F}_{impt1} \leftarrow \{\}$
18 **for** *each epoch* e **do**
19 $D_{(\theta_j,e)} = |\max_{b\in B_e} \mathcal{P}_{susp_{class}} - \max_{b\in B_e} \mathcal{P}_{clean_{class}}|$
 $= |\max_{b\in B_e} \Delta\theta^{(j)}_{s(e,b),c2(e,b)} - \max_{b\in B_e} \Delta\theta^{(j)}_{c1(e,b),c2(e,b)}|$;
20 $\mu = \frac{1}{m}\sum_{i=1}^{m} D_{(\theta_j,e)}$;
21 $\sigma = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(D_{(\theta_j,e)} - \mu)^2}$;
22 $\tau_1 = \mu + k \cdot \sigma$;
23 $\mathcal{F}_{impt1}[e] = \{i \mid D_{(\theta_j,e)} > \tau_1\}$
24 **end**
25 $\mathcal{F}_{impt1} = \sum_{e\in\mathcal{E}} \mathcal{F}_{impt1}[e]/\mathcal{E}$;
26 **return** $\mathcal{F}_{impt1}$

Tainted Batch Identification: For each batch $\mathcal{B}$ at epoch $\mathcal{E}$, batch-level provenance capture depends on the composition of suspect samples $\mathcal{B}_s$ (i.e., samples from $\mathcal{D}_u$) in $\mathcal{B}$ and their class label distribution. A batch $\mathcal{B}$ can contain all, some, or none of its samples from $\mathcal{D}_u$. If $\mathcal{B}$ contains at least one suspect, it is marked as tainted, which warrants provenance capture due to the potential influence of poisoning (line 3 of Algorithm 1). In contrast, if all the samples in $\mathcal{B}$ come from $\mathcal{D}_c$, capturing the provenance is unnecessary to avoid an imbalance in the provenance between $\mathcal{D}_c$ and $\mathcal{D}_u$.

Creating Matching Batch Variations: When $\mathcal{B}$ is tainted, before we capture the batch-level provenance that reflects the influence of suspect samples in $\mathcal{B}$ on model parameter update, we need to establish a fair ground for comparison by matching the data distribution of clean and suspicious samples within $\mathcal{B}$. We do so by sampling two subsets of clean samples from $\mathcal{B}$: $\mathcal{B}_{c1}$ and $\mathcal{B}_{c2}$, ensuring that both match $\mathcal{B}_s$ in sample size and class label distribution. If $\mathcal{B}_s$ mismatches $\mathcal{B}_{c1}$ or $\mathcal{B}_{c2}$ in size or class distribution, random samples are drawn from $\mathcal{D}_c$ to eliminate the mismatch. With the resulting three batch variations $\mathcal{B}_{c1}$, $\mathcal{B}_{c2}$, and $\mathcal{B}_s$, we exhaustively capture provenance data for all unique class labels in $\mathcal{B}_s$.

Parameter Updates and Provenance Capture: Using the model instance at batch $\mathcal{B}$, we compute model parameter updates for each batch variation, for each class in $\mathcal{B}_s$. As a result, we end up with three model parameter updates (lines 8-10 of Algorithm 1):
$\mathcal{U}_1 =$ `parameter-update`$(\mathcal{F}_{\mathcal{B}}, \mathcal{B}_{c1[class]})$
$\mathcal{U}_2 =$ `parameter-update`$(\mathcal{F}_{\mathcal{B}}, \mathcal{B}_{c2[class]})$
$\mathcal{U}_3 =$ `parameter-update`$(\mathcal{F}_{\mathcal{B}}, \mathcal{B}_{s[class]})$

We precisely track the provenance of tainted samples by computing the element-wise parameter update distances as clean batch provenance data ($\mathcal{P}_c$): `distance`$(\mathcal{U}_1, \mathcal{U}_2)$ and suspicious batch provenance data ($\mathcal{P}_s$): `distance`$(\mathcal{U}_1, \mathcal{U}_3)$ or `distance`$(\mathcal{U}_2, \mathcal{U}_3)$ (lines 11-12 of Algorithm 1). To simplify distance computation, we flatten the model parameters when computing the parameter update distances. This approach makes PoisonSpot applicable to various types of models, from convolutional to transformer-based models, as any complex model can be transformed to a one-dimensional array encapsulating all its weight parameters. For instance, flattened attention weights in the case of transformer-based models. This, in turn, makes PoisonSpot agnostic to data modality, as its flexibility in model selection ensures compatibility with any data type.

In line with our clean-label, targeted backdoor attacks threat model (§4.1), provenance data is captured separately for each class in $\mathcal{D}_{\mathrm{u}}$, with class-specific provenance denoted as $(\mathcal{P}_{\mathrm{s}}, \mathcal{P}_{\mathrm{c}})$.

$$\mathcal{P}_{\mathrm{s}} = \left\{\Delta\theta^{(j)}_{s(e,b),c2(e,b)}\right\}_{\substack{e=1,\ldots,E\\ b=1,\ldots,B_e\\ j=1,\ldots,m}} \quad \mathcal{P}_{\mathrm{c}} = \left\{\Delta\theta^{(j)}_{c1(e,b),c2(e,b)}\right\}_{\substack{e=1,\ldots,E\\ b=1,\ldots,B_e\\ j=1,\ldots,m}}$$

where, E is the total number of epochs; B_e is the number of tainted batches in epoch e; m is the number of parameters (dimension) of the model $\mathcal{F}$; $\Delta\theta^{(j)}_{s(e,b),c2(e,b)}$ is the parameter update difference between suspect (s) and clean ($c2$) batches for the jth model parameter from epoch e and tainted batch b; $\Delta\theta^{(j)}_{c1(e,b),c2(e,b)}$ is the update difference between clean batch pairs ($c1$ and $c2$) for the jth model parameter at epoch e and tainted batch b.

Training Provenance Pruning: The dimensions of $\mathcal{P}_c$ and $\mathcal{P}_s$ match the model parameter size m, often millions to billions for modern DNNs. Direct analysis of such large-scale provenance data is compute- and memory-intensive. To reduce this complexity, we perform training provenance pruning to identify the most relevant training provenance features, model parameters with the strongest signals of influence on updates.

Our intuition is that parameters affected by poisoning exhibit significant differences between `distance`$(\mathcal{U}_1, \mathcal{U}_2)$: parameter update differences between two clean-batch-based updates and `distance`$(\mathcal{U}_2, \mathcal{U}_3)$: parameter update differences between a clean-batch-based and suspect-batch-based update. For each parameter j in an epoch e, we compute the maximum parameter update difference between its clean parameter update difference and suspect parameter update difference. Using the maximal values of parameter updates ensures that the differences are not skewed by a large portion of clean samples in $\mathcal{B}_s$, which can occur when the percentage of poisons in $\mathcal{B}_s$ is small. Lines 19-22 of Algorithm 1 capture this step. Formally, we denote the maximum parameter update difference for j^{th} parameter at epoch e and batch b as:

$$D_{(\theta_j,e)} = |\max_{b\in B_e} \Delta\theta^{(j)}_{s(e,b),c2(e,b)} - \max_{b\in B_e} \Delta\theta^{(j)}_{c1(e,b),c2(e,b)}|$$

where, $\Delta\theta^{(j)}_{s(e,b),c2(e,b)}$ is the parameter update difference between a suspect batch s and clean batch $c2$ and $\Delta\theta^{(j)}_{c1(e,b),c2(e,b)}$ is the parameter update difference between two clean batches $c1$ and $c2$.

A training provenance feature j is selected if $D_{(\theta_j,e)} > \tau_1$, where τ_1 is empirically determined based on the mean (μ) and standard deviation (σ) of $D_{(\theta_j,e)}$ values. It is computed as $\tau_1 = \mu + k \cdot \sigma$, where k is a hyperparameter that controls the sensitivity of feature

Algorithm 2: Sample-Level Training Provenance Capture

Input: $\mathcal{D}_c$, $\mathcal{F}$, $\mathcal{D}_u$
Output: $\mathcal{P}_{clean,class} = \{\}$, $\mathcal{P}_{susp,class} = \{\}$ for each class in $\mathcal{D}_u$
1 **for** *each epoch* $\mathcal{E}$ **do**
2 **for** *each batch* $\mathcal{B}$ **do**
3 $\mathcal{B}_s \leftarrow \mathcal{B} \cap \mathcal{D}_u$;
4 **if** $\mathcal{B}_s \neq \emptyset$ **then**
5 $\mathcal{B}_{c1} \leftarrow$ `sample-from`($\mathcal{B}$); `// or sample-from`($\mathcal{D}_c$)
6 $\mathcal{B}_{c2} \leftarrow$ `sample-from`($\mathcal{B}$); `// or sample-from`($\mathcal{D}_c$)
7 **for** *each* $class$ *in* $\mathcal{B}_s$ **do**
8 $\mathcal{B}_{all} \leftarrow \mathcal{B}_{c1[class]} \cup \mathcal{B}_{c2[class]} \cup \mathcal{B}_{s[class]}$;
9 Initialize $\mathcal{U}_{2,\text{sum}}$;
10 **for** *each sample* $x \in \mathcal{B}_{all}$ **do**
11 **if** $x \in \mathcal{B}_{c1[class]}$ **then**
12 $\mathcal{U}_{1,x} \leftarrow$ `parameter-update`($\mathcal{F}_{\mathcal{B}}, x$);
13 **else**
14 **if** $x \in \mathcal{B}_{c2[class]}$ **then**
15 $\mathcal{U}_{2,x} \leftarrow$ `parameter-update`($\mathcal{F}_{\mathcal{B}}, x$);
16 $\mathcal{U}_{2,\text{sum}} \leftarrow \mathcal{U}_{2,\text{sum}} + \mathcal{U}_{2,x}$;
17 **else**
18 $\mathcal{U}_{3,x} \leftarrow$ `parameter-update`($\mathcal{F}_{\mathcal{B}}, x$);
19 **end**
20 **end**
21 **end**
22 $\mathcal{U}_{2_avg} \leftarrow \mathcal{U}_{2,\text{sum}}/|\mathcal{B}_{c2[class]}|$;
23 **for** *each sample* $x \in \mathcal{B}_{all}$ **do**
24 **if** $x \in \mathcal{B}_{c1[class]}$ **then**
25 $\mathcal{P}_{clean,class}[x, epoch] \leftarrow$ `distance`($\mathcal{U}_{1,x}, \mathcal{U}_{2_avg}$);
26 **else**
27 **if** $x \in \mathcal{B}_{s[class]}$ **then**
28 $\mathcal{P}_{susp,class}[x, epoch] \leftarrow$ `distance`($\mathcal{U}_{3,x}, \mathcal{U}_{2_avg}$);
29 **end**
30 **end**
31 **end**
32 **end**
33 **end**
34 **end**
35 **end**
36 **return** $\mathcal{P}_{susp,class}, \mathcal{P}_{clean,class}$ *for each class in* $\mathcal{D}_u$;

selection. Here, we note that while k is a hyperparameter, by selecting features with parameter update difference values k standard deviations away from μ, our approach favors the most relevant set of training provenance features likely to be highly affected by poisonous samples. k should be set to a low value to include more features to further refine. Finally, the features with $D_{(\theta_j,e)} > \tau_1$ form the set of the most relevant training provenance features $\mathcal{F}_{impt1}$, which serves as input for the sample-level provenance analysis.

6.1.2 Sample-Level Training Provenance Capture

Sample-level provenance capture focuses on the individual influence of each sample on the model's parameter updates. As in batch-level provenance capture, we identify tainted samples by their membership in $\mathcal{D}_u$. Unlike batch-level provenance, matching batch variations are created using single data points from $\mathcal{B}_{c1}$, $\mathcal{B}_{c2}$, and $\mathcal{B}_s$ individually for each class label.

We then capture the model parameter updates for each sample to detect their individual impact. Because the impact of provenance data is captured as the difference between two successive model parameter updates, we take the difference between each of the model parameter updates for each sample x in $\mathcal{U}_{3,x}$ and $\mathcal{U}_{1,x}$, and the average of the model parameter updates for one of the clean

Algorithm 3: Custom Self-Training

Input: $\mathcal{P}_c$, $\mathcal{P}_s$, r (Suspicious-to-clean ratio), γ (Score threshold), max_iters (Max iterations)
Output: Final classifier $\mathcal{M}_{final}$
$\mathcal{P}_l \leftarrow \emptyset$;
$\mathcal{P}_u \leftarrow \mathcal{P}_s$;
$\mathcal{F}_0 \leftarrow \text{Train}(\mathcal{P}_s \cup \mathcal{P}_c)$;
for $i \leftarrow 1$ **to** max_iters **do**
 $S \leftarrow \mathcal{F}_{i-1}.\text{predict_proba}(\mathcal{P}_u)[:, 1]$;
 $H \leftarrow \{ x \in \mathcal{P}_u : S[x] \geq \gamma \}$;
 if $H = \emptyset$ **then**
 break
 end
 $\mathcal{P}_l \leftarrow \mathcal{P}_l \cup H$;
 $\mathcal{P}_u \leftarrow \mathcal{P}_u \setminus H$;
 $\mathcal{P}^{(i)}_{\text{train}} \leftarrow \mathcal{P}_l \cup \text{Sample}(\mathcal{P}_c, r \cdot |\mathcal{P}_l|)$;
 $\mathcal{F}_i \leftarrow \text{Train}(\mathcal{P}^{(i)}_{\text{train}})$;
end
$\mathcal{M}_{final} \leftarrow \mathcal{F}_i$;
return $\mathcal{M}_{final}$;

batches $\mathcal{U}_{2_avg} = \frac{\mathcal{U}_{2,\text{sum}}}{|\mathcal{B}_{c2}|}$ (line 22 of Algorithm 2) to serve as our provenance data, $\mathcal{P}_c$ and $\mathcal{P}_s$, respectively. We take the average to avoid outliers that influence the model parameter updates.

The sample-level provenance data is similar to batch-level provenance data except that the rows of the provenance data now represent individual sample impact as opposed to batches. Therefore, for each suspected sample sus_k from $\mathcal{D}_u$ with sample IDk, clean reference sample c_k with IDk from $\mathcal{D}_c$, and the number of fine-tuning epochs ep, the provenance data $\mathcal{P}_s$, and $\mathcal{P}_c$ is expressed as follows:

$$\mathcal{P}_{\text{c}} = \left\{ \Delta\theta^j_{(n,\text{c}_k)} \right\}_{\substack{n=1,\ldots,E \\ k=1,\ldots,n_c \\ j=1,\ldots,m}} \quad \mathcal{P}_{\text{s}} = \left\{ \Delta\theta^j_{(n,\text{sus}_k)} \right\}_{\substack{n=1,\ldots,E \\ k=1,\ldots,n_s \\ j=1,\ldots,m}}$$

where, E is the number of fine-tuning epochs; n_s is the total number of suspected samples in $\mathcal{D}_{\text{u}}$; n_c is the total number of clean reference samples, a subset of $\mathcal{D}_{\text{c}}$; $\Delta\theta^j_{(n,\text{sus}_k)}$ is the change in the jth model parameter during epoch n training on a suspect sample sus_k (with ID k); and $\Delta\theta^j_{(n,\text{c}_k)}$ is the change in the jth model parameter during epoch n by training on a clean sample c_k (with ID k).

6.2 Training Provenance Analysis

Training provenance analysis (Phase II in Figure 2) refines feature selection and computes poisoning scores to effectively identify and isolate tainted samples. This involves two main steps, (1) *feature refinement* to remove spurious or noisy features and (2) *poisoning scoring* which assigns a poisoning score to each sample.

Provenance Feature Refinement: This step employs leave-one-group-out test on $\mathcal{P}_{clean}$ and $\mathcal{P}_{susp}$ to further refine features based on feature importance scores, retaining those that contribute greatly to poisoning. Then, a final leave-one-group-out scoring is performed using the refined features to obtain poisoning scores.

We refine the relevant features by doing a leave-one-group-out testing on $\mathcal{P}_c$ and $\mathcal{P}_s$: this is shown in Algorithm 4. Further refinement is needed because some features from the first pruning in Phase I may have high differences randomly which might not be related to poisoning. Further refinement using feature importance from a model would clear those features by retaining the ones that

are relevant for the distinction between clean and poisonous samples. To perform cross-validation, we label $\mathcal{P}_c$ as clean and $\mathcal{P}_s$ as poisonous and divide our training data into some number of groups n_splits, where n_splits is set as a hyperparameter.

For each group, we perform custom self-training [2] (Algorithm 3) on all other groups and test it on the excluded group. Self-training first starts by training a model $\mathcal{F}_0$ on $(\mathcal{P}_s \cup \mathcal{P}_c)$. At each iteration $i \leq \max_iters$, the model $\mathcal{F}_{i-1}$ computes poisoning probabilities on the unlabeled set $\mathcal{P}_u$ and extracts the high-confidence subset H. It then removes H from $\mathcal{P}_u$, adds it to the labeled set $\mathcal{P}_l$, rebuilds the training set by combining $\mathcal{P}_l$ with $r \cdot |\mathcal{P}_l|$ clean examples sampled from $\mathcal{P}_c$, and retrains to obtain $\mathcal{F}_i$. We down-sample $\mathcal{P}_c$ for balanced training. The loop ends when H is empty or $i = \max_iters$, resulting in the final classifier $\mathcal{M}_{\text{final}} = \mathcal{F}_i$. This method avoids overfitting by removing most of the falsely labeled clean points from $\mathcal{P}_s$ to result in better-refined provenance data $\mathcal{P}_l$. We use a Random Forest model with restricted maximum depth because its use of bagging and majority voting makes it less prone to overfitting than deep learning models [17]. Overfitting should be avoided, as some samples labeled as poisonous may actually be clean. This model choice is validated in §7.2: **A3**.

As we test on each group, we use a Random Forest feature importance score to get the feature importance for each testing group, and we average them to get a single feature importance score for each of our relevant set of features. We use average to make sure that the features are of high importance between the different cross-validation groups. Then, we can choose the most important set of features taking those that have an importance score greater than a feature importance threshold τ_2. Using our new set of important features $\mathcal{F}_{impt2}$, we further refine our provenance data to obtain $\mathcal{P}_c = \{\Delta\theta^{(j)}_{i,cc} \mid i \in \mathcal{F}_{impt2}\}$ and $\mathcal{P}_s = \{\Delta\theta^{(j)}_{i,sc} \mid i \in \mathcal{F}_{impt2}\}$.

Poisoning Scoring: Using the final set of relevant features, we perform a leave-one-group-out scoring to compute poisoning scores $\mathcal{PS}$ (lines 15-19 of Algorithm 4). This is done in the same manner as before except that we extract the model prediction probabilities for being poisonous to use as the poisoning score for each clean and suspicious batch. The number of groups for the splits m_{splits} is set as a hyperparameter as before. By combining feature refinement and scoring, we minimize false positives and false negatives.

Algorithm 4: Training Provenance Analysis

Input: $\mathcal{P}_c$, $\mathcal{P}_s$ **Output:** $\mathcal{PS}$ $\mathcal{P} = \mathcal{P}_c \cup \mathcal{P}_s$

$n_{splits} \leftarrow 5$ ▹ n_splits is a hyperparameter
Split $\mathcal{P}$ into n_{splits} groups for cross-validation;
  Initialize $F_i \leftarrow \{\}$;
  for *each group g in* $\mathcal{P}$ **do**
    Self training(()$\mathcal{P} \setminus \{g\}$);
    Test(()$\mathcal{F}$, g);
    $F_i[g_{\text{num}}] \leftarrow$ FeatureImportances(()$\mathcal{F}$) ; // g_{num}: index of g
  end
  $F_{i_avg} \leftarrow \frac{1}{n_{splits}} \sum_{i=1}^{n_{splits}} F_i[i]$;
  $\mathcal{F}_{impt2} \leftarrow \{i \mid F_{i_avg}[\theta_i] > \tau_2\}$;
  $\mathcal{P} = \mathcal{P}_c[\mathcal{F}_{impt2}] \cup \mathcal{P}_s[\mathcal{F}_{impt2}]$;
  $m_{splits} \leftarrow 5$; // Number of splits is a hyperparameter
1 Split $\mathcal{P}$ into m_{splits} groups for cross-validation;
2 Initialize $\mathcal{PS} \leftarrow \{\}$;
3 **for** *each g in* $\mathcal{P}$ **do**
4 **for** *each* (x, e) *in g* **do**
5 $\mathcal{PS}[(x, e)] \leftarrow$ Prediction Probability(()F, $g[(x, e)]$)
6 **end**
7 **end**
8 **return** $\mathcal{PS}$;

Algorithm 5: Poisoning Score Aggregation & Thresholding

Input: $\mathcal{PS}$
Output: *Poisonous_Samples* (List of Poisonous Samples)
1 **Step 1: Compute aggregate poisoning score**;
2 Initialize $PS_{agg} \leftarrow \{\}$;
3 **for** *each* $(x, e) \in \mathcal{PS}$ **do**
4 $PS_{agg}[x] \leftarrow \frac{1}{\mathcal{E}} \sum_{i=1}^{\mathcal{E}} \mathcal{PS}[(x, i)]$; // Avg. over all epochs
5 **end**
6 **Step 2: Threshold Determination**;
7 $\tau_3 \leftarrow$ DetermineThreshold(()PS_{agg}, method) ; // method: K-means|Gaussian
8 *Poisonous_Samples* $\leftarrow \emptyset$;
9 **for** *each* $x \in PS_{agg}$ **do**
10 **if** $PS_{agg}[x] > \tau_3$ **then**
11 *Poisonous_Samples* $\leftarrow$ *Poisonous_Samples* $\cup \{x\}$;
12 **end**
13 **end**
14 **return** *Poisonous_Samples*;

6.3 Poisoning Score Aggregation and Thresholding

Aggregation: After obtaining the poisoning scores for each sample across $\mathcal{E}$ training epochs, the next step is to calculate a single aggregate score per sample. The reason for aggregation is twofold: (1) the provenance is captured in later epochs when the model parameters have mostly stabilized and (2) variations in parameter updates across epochs are minor and hence should be weighted equally. Lines 1-5 of Algorithm 5 show the details of how the average poisoning score is calculated per sample. Taking the average ensures a balanced and robust score that reflects the overall impact of a sample throughout the training process.

Thresholding: Once each sample has a single aggregate poisoning score, the final task is to identify a threshold (τ_3: line 7 of Algorithm 5) to separate clean and poisonous samples. This step relies on clustering methods to exploit the expected bimodal distribution of scores. To determine the threshold τ_3, we use two complementary clustering methods: Gaussian Mixture Model (GMM) and K-means clustering. GMM is suitable for large sample sizes, where scores are expected to follow a Gaussian distribution. The intersection of the two Gaussian curves is used as the threshold. K-means clustering is effective for smaller datasets. In line 7 of Algorithm 5, the clustering *method* is set as a hyperparameter. Alternatively, the clustering method can be selected dynamically based on each method's TPR and FPR, to prioritize a clustering method with a high TPR at low FPR to minimize the risk of retaining poisonous samples. We then eliminate samples with a poisoning score greater than a threshold τ_3 (lines 8-13 of Algorithm 5). As we show in our evaluation in §7, by employing these methods, PoisonSpot effectively distinguishes between clean and poisonous samples, ensuring accurate identification and elimination of tainted data points.

7 Evaluation

Our evaluation is guided by the following research questions:
• **RQ1 (§7.2)**: Are PoisonSpot's core assumptions valid?
• **RQ2 (§7.3)**: How effective is PoisonSpot across (a) datasets, (b) attacks, (c) poison ratios, and (d) training settings (retraining from scratch or fine-tuning with suspect samples)?
• **RQ3 (§7.4)**: How does PoisonSpot compare with state-of-the-art backdoor detection approaches?
• **RQ4 (§7.5)**: How resilient is PoisonSpot against adaptive attacks?
• **RQ5 (§7.6)**: Is PoisonSpot practical in terms of computational overhead and memory footprint?

7.1 Experimental Setup

Datasets and Models. We consider 3 image classification datasets: CIFAR-10 [21], ImageNet-100 [12], and STL-10 [11]. Details of each dataset, model, and hyperparameters appear in the Appendix (§8.3).

Attacks. We consider 4 clean-label backdoor attacks that align with our threat model (§4.1) and used in the recent literature. They are Hidden-Trigger [34], Label-Consistent [40], Sleeper-Agent [37], and Narcissus [46]. On CIFAR-10, for Label-Consistent, we use a trigger placed at all four corners with no reduction in visibility of the trigger. We use a perturbation budget of $\frac{16}{255}$ for all attacks except for Label-Consistent where we use a perturbation budget of $\frac{8}{255}$. We use fixed triggers for Sleeper-Agent and Hidden-Trigger.

Baseline Defense. We compare PoisonSpot with ASSET [31]: a state-of-the-art clean-label backdoor attack detection method. ASSET is reported to be superior to existing defenses in terms of its consistent success rate with different attacks and its robustness to variations in poison ratios. It is also the only defense to detect the state-of-the-art clean-label attack: Narcissus [46]. We note that our attempt to reproduce the ASSET code was not successful due to errors in the official implementation[2] of ASSET. We repeatedly contacted the authors but we did not get a response. Consequently, we compare our results with the results reported in the ASSET paper. To ensure a fair comparison, we matched our setup with ASSET's training parameters as closely as possible based on the training setup described in the paper and their code.

Metrics. As in ASSET [31], we report both *upstream evaluation* and *downstream evaluation*. Upstream evaluation assesses how accurately poisoned samples are detected while downstream evaluation assesses how the model performs after removing poison samples. We measure the **TPR** and **FPR** for upstream evaluation, and Attack Success Rate (**ASR**) and model's test accuracy (**ACC**) for downstream evaluation. Although a high TPR at low FPR is desirable, the ultimate indicator of PoisonSpot's effectiveness is the downstream evaluation results, where we aim for a low ASR and downstream **ACC** close to the original (**ACC**). We use two variations of poison ratio (**PR**): $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$ and $\mathbf{PR}(\mathcal{D}_u)$. $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$ represents the percentage of poisonous samples in the entire training set, including the clean dataset $\mathcal{D}_c$ and the unknown dataset $\mathcal{D}_u$, while $\mathbf{PR}(\mathcal{D}_u)$ represents the percentage of poisonous samples in the unknown dataset $\mathcal{D}_u$. For example, for a CIFAR-10 dataset containing 50,000 images where 48,000 images belong to $\mathcal{D}_c$, 2000 belong to $\mathcal{D}_u$, and 500 images within $\mathcal{D}_u$ are poisonous, $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$ is 1% ($\frac{500}{50000}$) and $\mathbf{PR}(\mathcal{D}_u)$ is 25% ($\frac{500}{2000}$). These ratios are key metrics for measuring PoisonSpot's robustness to changes in poison ratio..

[2]https://github.com/reds-lab/ASSET

7.2 RQ1: Validating PoisonSpot Assumptions

We first assess the validity of assumptions A1–A5 on CIFAR-10.

A1: To validate that clean parameter updates differ significantly from poisoned parameter updates, we performed cross-validation on a model trained on the clean and suspected batch-level provenance data. High cross-validation accuracy suggests that the model can reliably distinguish between clean and poisonous samples based on their model parameter update differences. The results shown in Table 2 suggest that cross-validation accuracy exceeds 95 % for all four attack types which **confirms that clean and poisoned parameter updates are significantly different**.

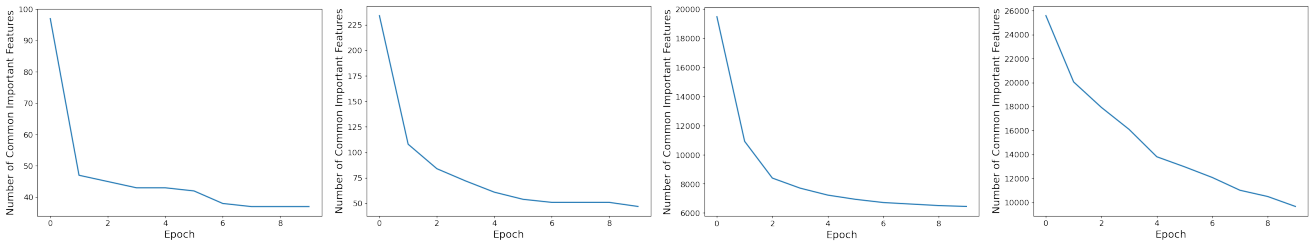
A2: To show that a consistent set of model parameters are uniquely affected by poisoned samples, we analyze the overlap of the selected important provenance features ($\mathcal{F}_{impt1}$) across epochs. For each epoch, we record the important features and then compute the intersection with those from all previous epochs. As shown in Figures 3a–3d, the number of common features is high initially, but stabilizes in later epochs, indicating that **there is a core subset of parameters that are consistently influenced by poisoning**. We observe the same pattern across various attacks except Hidden-Trigger, which exhibits a high and steadily decreasing feature set. This is due to the low model accuracy we used for the Hidden-Trigger attack ($\approx$ 70%) which produces larger parameter updates and thus causes unnecessary features to be selected which will be refined in the second phase. We also observe that the number of features varies depending on the type of attack.

A stronger proof of a consistent important feature set is shown by the stability of the second-phase selection process ($\mathcal{F}_{\text{impt}_2}$), as shown in Table 14 (Appendix: §8.1). Although the number of first-phase features ($\mathcal{F}_{\text{impt}_1}$) varies with k, the number of second-phase features ($\mathcal{F}_{\text{impt}_2}$) remains nearly identical, **further confirming the existence of a core set of parameters affected by poisoning**.

Attack	$\mathbf{PR}(D_c \cup D_u)$ (%)	$\mathbf{PR}(D_u)$ (%)	CV Accuracy (%)
Label-Consistent [40]	1	50	99.01
Narcissus [46]	1	50	100.00
Sleeper-Agent [37]	5	50	99.11
Hidden-Trigger [34]	5	50	95.78

Table 2: A1: Cross-validation accuracy on batch-level provenance data across various attacks.

A3: As in **A1**, but with sample-level provenance data, we show that differences in parameter updates between poisoned and clean samples remain evident when parameter updates are captured on a per-sample basis. We set $\mathbf{PR}(D_u)$ to 10% to demonstrate that, even at lower poison ratios in the suspected dataset, cross-validation between clean and suspected samples still effectively detects poisoned samples. Table 2 shows the cross-validation accuracy of identifying poisoned samples in the sample-level provenance data is greater than 90% which **confirms that parameter update differences are also evident at the sample-level** which enables PoisonSpot to precisely spot poisoned samples.



(a) Attack: Label-Consistent **(b) Attack: Narcissus** **(c) Attack: Sleeper-Agent** **(d) Attack: Hidden-Trigger**

Figure 3: Number of common important features ($\mathcal{F}_{\text{impt1}}$) per epoch.

Attack	PR($D_c \cup D_u$) (%)	PR(D_u) (%)	CV Accuracy (%)
Label-Consistent [40]	1	10	100.00
Narcissus [46]	1	10	100.00
Sleeper-Agent [37]	1	10	92.07
Hidden-Trigger [34]	5	50	90.48

Table 3: A3: Cross-validation accuracy on sample-level provenance data across different attacks.

A4: We compare five models: MLP, a simple DNN, a default Random Forest, a Random Forest with max depth = 10 (to avoid overfitting), and a Random Forest with self-training (Algorithm 3: $r = 3$, $max_iters = 1$, $\gamma = 0.7$) using cross-validation accuracy on Sleeper-Agent with PR($D_c \cup D_u$) = 1% and PR(D_u)= 10%. This comparison shows how detection performance varies with model choice and how overfitting impacts the results. **Models that overfit perform poorly when suspected dataset labels are noisy**, especially at low PR(D_u). Therefore, we require simpler models that can reliably distinguish between clean and poisoned parameter features. Table 4 reports each model's TPR and FPR. Random Forest with max depth = 10 and with self-training achieve the best balance of high TPR and low FPR. In contrast, default Random Forest, DNN, and MLP all exhibit high FPR or low TPR because they are prone to overfitting. A semi-supervised self-training approach with Random Forest model mitigates this issue and proves as the most effective.

Model Type	TPR (%)	FPR (%)
Random Forest (Self Training)	94.0	1.71
Random Forest (Max Depth = 10)	91.4	1.57
Random Forest (Default)	73.2	0.06
DNN	92.2	57.06
MLP	94.4	38.38

Table 4: Comparison of cross-validation of 5 models based on TPR and FPR for Sleeper-Agent with PR($D_c \cup D_u$) = 10%**.**

A5: Table 8 (Narcissus column) shows that, when the untrusted set $\mathcal{D}_u$ contains at least 5% poisoned samples corresponding to a 0.05% overall poison ratio in $\mathcal{D}_c \cup \mathcal{D}_u$ PoisonSpot achieves a TPR of 100% and a FPR of 0%.

Attack	Label-Consistent [40]		Sleeper-Agent [37]		Narcissus [46]	
PR($\mathcal{D}_c \cup \mathcal{D}_u$) , PR($\mathcal{D}_u$) , TC	1%, 10%, Class 2		1%, 10%, Class 1		0.05%, 10%, Class 2	
Upstream Evaluation						
	TPR	FPR	TPR	FPR	TPR	FPR
ASSET ([31])	96.2	0.75	96.6	0.39	92.0	0.34
PoisonSpot	100	0	93.6	1.76	100	0
Downstream Evaluation						
	ASR	ACC	ASR	ACC	ASR	ACC
No Defense (ours)	98.71	95.24	99.7	92.60	99.93	95.41
No Defense ([31])	100	94.7	76.7	94.4	99.7	94.9
ASSET ([31])	1.16	94.9 (↑0.2)	1.14	94.4 (-)	9.68	94.9 (-)
PoisonSpot	0.32	95.58 (↑0.34)	2.4	92.32 (↓0.28)	1.22	95.38 (↓0.03)

Table 5: Training from Scratch comparison of PoisonSpot with ASSET under **RQ2** and **RQ3** on CIFAR-10 using upstream and downstream evaluations. **TC**: target class. ↑ denotes increase in accuracy while ↓ denotes decrease in accuracy.

Attack	Hidden-Trigger [34]		Sleeper-Agent [37]		Avg. TPR	Worst FPR
PR($\mathcal{D}_c \cup \mathcal{D}_u$) , PR($\mathcal{D}_u$) , TC , SC	5%, 50%, Class 4, Class 3		5%, 50%, Class 1, Class 0		Avg.	Worst
Upstream Evaluation						
	TPR	FPR	TPR	FPR	Avg. TPR	Worst FPR
ASSET	98.7	1.03	95.2	0.51	96.95	1.03
PoisonSpot	98.52	0	92.2	0	95.36	0
Downstream Evaluation						
	ASR	ACC	ASR	ACC	Avg. ASR	Worst ACC
No Defense (ASSET [31])	82.20	71.53	92.00	90.14	87.10	71.53
No Defense (PoisonSpot)	59.90	73.07	100.0	90.83	79.95	73.07
ASSET ([31])	3.4	72.8 (-)	8.40	92.3 (-)	5.9	72.8
PoisonSpot	2.6	73.75 (↑0.68)	0.4	91.69 (↑1.55)	1.5	73.75

Table 6: Fine-tuning comparison of PoisonSpot with ASSET under **RQ2** and **RQ3** on CIFAR-10 using upstream and downstream evaluations. **SC**: source class.

7.3 RQ2: Overall Effectiveness

Training From Scratch with Suspect Samples: All attacks are evaluated under the training from scratch scenario, except Hidden-Trigger, as it only works with fine-tuning. Similar to ASSET [31], we use 1% **PR**($\mathcal{D}_c \cup \mathcal{D}_u$) for Label-Consistent and Sleeper-Agent, and 0.05% **PR**($\mathcal{D}_c \cup \mathcal{D}_u$) for Narcissus. For **PR**($\mathcal{D}_u$), we set it to 10% for all the attacks. The target label used for both Label-Consistent and Narcissus is class 2, while we used class 1 as the target class for Sleeper-Agent with source class 0. Using a different target class for Sleeper-Agent demonstrates that PoisonSpot is flexible across target classes. The results for training from scratch scenario are shown in Table 5 for CIFAR-10. Against Label-Consistent and Narcissus attacks, PoisonSpot performs incredibly well with 100% TPR and 0% FPR. This is mainly because the poisonous samples for both attacks have a strong poisoning influence on the model parameter update. Additionally, we fix the trigger in the same place for both attacks,

Attack	Label-Consistent [40]				Narcissus [46]				Sleeper-Agent [37]			
TC	Class 2				Class 2				Class 1			
$\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)(\mathbf{PR}(\mathcal{D}_u))$	TPR	FPR	ACC	ASR	TPR	FPR	ACC	ASR	TPR	FPR	ACC	ASR
1% (50%)	100	0	95.58	-	100	0	95.38	-	98.6	0.2	92.44	0.2
2% (50%)	100	0	95.34	-	100	0	95.30	-	99.5	0.2	92.21	0.4
3% (50%)	100	0	95.09	-	100	0	95.28	-	96.13	1.3	92.05	0.2

Table 7: Effect of varying poison ratios in the training set ($\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$) under RQ2 on CIFAR-10 across attacks.

Attack	Label-Consistent [40]				Narcissus [46]				Sleeper-Agent [37]			
$\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$, TC	1%, Class 2				1%, Class 2				1%, Class 1			
$\mathbf{PR}(\mathcal{D}_u)$	TPR	FPR	ACC	ASR	TPR	FPR	ACC	ASR	TPR	FPR	ACC	ASR
5% (0.05% $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$)	-	-	-	-	100	0	95.38	-	-	-	-	-
10%	100	0.82	95.15	-	100	0.48	95.30	-	93.6	1.76	92.32	2.4
25%	100	0	95.58	-	100	0	95.38	-	95.6	0.13	91.96	0.8
50%	100	0	95.58	-	100	0	95.38	-	98.6	0.2	92.44	0.6
75%	100	0.05	94.75	-	100	0	95.38	-	98.6	0.0	91.71	0.6
100%	100	0	95.58	-	100	0	95.38	-	99	0	92.14	0.10

Table 8: Effect of varying poison ratio in the unknown dataset ($\mathbf{PR}(\mathcal{D}_u)$) under RQ2 on CIFAR-10 across attacks.

so the model uses a common set of pixels to predict the target class which means a few parameters would be consistently updated making it easier for PoisonSpot to detect poisoned samples. Sleeper-Agent, on the other hand, uses gradient alignment to solve a bi-level optimization problem to hide the triggers. This causes the poisoned samples to have varying visibility and poisoning ability making the set of parameters updated not as consistent as Label-Consistent and Narcissus. This means the impact of individual poisonous samples on the model parameter updates is not uniformly strong, which makes it relatively harder for PoisonSpot to detect all the poisonous images. However, when we observe the downstream evaluation results, the remaining undetected samples have a minimal effect on the poisonous model parameter updates, which is reflected by the low downstream ASR. For instance, the downstream ASR is 0.6% for Sleeper-Agent, although PoisonSpot missed almost 8% of the poisonous samples. ***Therefore, PoisonSpot successfully detects backdoors inserted in the training-from-scratch scenario.***

Fine-tuning with Suspect Samples: For fine-tuning, we evaluate PoisonSpot against Hidden-Trigger [34] and Sleeper-Agent [37]. The results are shown in Table 6. We fine-tuned a clean ResNet-18 model (trained for 200 epochs) for 10 epochs. The target and source classes for Hidden-Trigger are 4 and 3, respectively, and for Sleeper-Agent, they are 1 and 0, respectively. $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$ is 5% and $\mathbf{PR}(\mathcal{D}_u)$ is 50% for both attacks. Since the size of $\mathcal{D}_u$ is 10%, it means the entire target class is suspected. We used the clean target samples from the test set as our clean reference samples to capture the provenance data. Our experiments cover both fine-tuning cases where all model parameters are fine-tuned and the case where only last layer is fine-tuned. For Hidden-Trigger, only the last layer is fine-tuned, but for Sleeper-Agent, the whole layers are updated. Regardless, PoisonSpot works well on both cases as the downstream ASR is below 10%, which implies an unsuccessful attack. We further validate PoisonSpot on two additional datasets: ImageNet-100 [12] and STL-10 [11] dataset (details in Appendix 8.2: Table 16). ***Therefore, PoisonSpot is equally effective at precisely detecting backdoors inserted during fine-tuning***.

Impact of Varying Poison Ratio: Table 7 shows results of robustness of PoisonSpot against changes in $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$ for the training dataset using Label-Consistent [40], Narcissus [46], and Sleeper-Agent [37] on CIFAR-10. We vary $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$ from 1% to 3%. PoisonSpot tracks the provenance data for each class, where for targeted backdoor attacks performed simultaneously on different target classes, a 3% $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$ can be theoretically stretched to 30% when all 10 classes are targeted. In §7.4, we increase this poison ratio further to 10% which is the maximum we can reach because each target class is 10% of the training set. We do this to challenge PoisonSpot with higher poison ratios. Based on the results for Label-Consistent and Narcissus, we can see that the change in $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$ does not have an effect on PoisonSpot's performance as the poisoned samples were accurately identified for all variations of $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$. For Sleeper-Agent, we can also see the results are fairly consistent despite the changes in $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$. We then assess PoisonSpot's robustness against variations in $\mathbf{PR}_{\mathcal{D}u}$. The results are shown in Table 8. The change in $\mathbf{PR}_{\mathcal{D}u}$ had a minimal effect for Label-Consistent and Narcissus attacks except for lower $\mathbf{PR}_{\mathcal{D}u}$ of 10% where the **FPR** was a little higher. We can also see for Sleeper-Agent that ***the higher the $PR_{\mathcal{D}u}$, the higher the TPR and the lower the FPR***. However, we tested the limit upto to 10% by making the whole target class suspected and PoisonSpot still successfully defends the attacks. ***Overall, all our results suggest that varying the poison ratio has a negligible effect on the performance of PoisonSpot***.

7.4 RQ3: Comparison with State-of-the-Art

From our training-from-scratch results in Table 5, PoisonSpot outperforms ASSET in the upstream evaluation both in terms of **TPR** and **FPR** except for **TPR** for Sleeper-Agent . However, the downstream evaluation of Sleeper-Agent showed success as PoisonSpot has eliminated the most poisonous ones and the remaining ones have minimal poisoning effect. ***PoisonSpot outperforms ASSET also in the downstream evaluation in terms of downstream ASR expect for Sleeper-Agent.*** PoisonSpot has a slight decrease for Narcissus and Sleeper agent for the downstream accuracy but it is not significant as the change was very minimal.

For fine-tuning on CIFAR-10, PoisonSpot performs better than ASSET in the upstream evaluation with 0% **FPR** for both attacks, but ASSET performs better for Sleeper-Agent with higher **TPR**. However, this does not matter because PoisonSpot outperforms ASSET slightly in the downstream evaluation as it has lower ASR for both attacks. In terms of the downstream **ACC**, ASSET maintains steady accuracy, while PoisonSpot has a slight increase in accuracy for both Hidden-Trigger and Sleeper-Agent. Although both the evaluations for PoisonSpot and ASSET can be deemed success, we observe that PoisonSpot performs better for CIFAR-10 due to lower downstream **ASR**. For the STL-10 dataset (Appendix: 8.2), PoisonSpot significantly outperforms ASSET with ≈10% difference in **TPR** and ≥15% difference in downstream ASR for Sleeper-Agent. PoisonSpot maintained less than 1% ASR for both attacks which entails a failed attack. ***Overall, PoisonSpot outperforms ASSET for fine-tuning across different datasets and attacks***.

7.5 RQ4: Resilience to Adaptive Attacks

We now evaluate PoisonSpot for its resilience against adaptive attacks which exploit perturbation strength, attack diversity, and variation in poison ratio.

Attack	Label-Consistent [40]		Narcissus [46]	
	($\epsilon = 4/255$ vis = 32)	($\epsilon = 2/255$, vis = 32)	($\epsilon = 6/255$)	($\epsilon = 8/255$)
PR($\mathcal{D}_c \cup \mathcal{D}_u$) , PR($\mathcal{D}_u$) , TC	1%, 50%, Class 2		1%, 50%, Class 2	
Original ACC	95.27	95.17	95.14	95.22
Original ASR	31.21	78.63	34.6	79.21
TPR	100	99	75.2	90.6
FPR	0	0	3.2	4.8
ASR	0.43	0.54	6.17	4.91
ACC	95.53 (↑0.26)	94.96 (↓0.21)	95.30 (↑0.16)	95.24 (↑0.02)

Table 9: Resilience to weaker perturbation-based backdoor triggers. vis is the visibility parameter for Label-Consistent.

Attack	Label-Consistent[40]	Narcissus[46]
PR($\mathcal{D}_c \cup \mathcal{D}_u$) , PR($\mathcal{D}_u$) , TC	1%, 50%, Class 2	0.05%, 50%, Class 2
Original ACC	95.49	
Original ASR (LC)	97.80	
Original ASR (N)	100.0	
TPR	100	
FPR	0	
ACC	95.37 (↓0.12)	

Table 10: Resilience to mixed triggers (2 attacks).

Attack	Label-Consistent [40]	Narcissus [46]	Sleeper-Agent [37]
PR($\mathcal{D}_c \cup \mathcal{D}_u$) , PR($\mathcal{D}_u$) , TC	1%, 50%, Class 2	0.05%, 50%, Class 2	1%, 50%, Class 2
Original ACC	89.30		
Original ASR (LC)	90.68		
Original ASR (N)	100.00		
Original ASR (SA)	79.70		
TPR (LC)	100		
TPR (N)	100		
TPR (SA)	97.6		
FPR	0.29		
ASR (SA)	0.9		
ACC	88.68 ((↓0.62))		

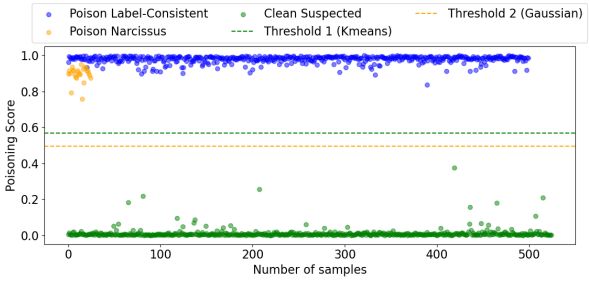
Table 11: Resilience to mixed triggers (3 Attacks).

Weak Perturbation-based Trigger. When the backdoor trigger insertion strategy involves mild manipulations, meaning that the attack success rate is significant but still low, the effect on model parameter updates is not as significant as when using a strong attack where the trigger carries strong signals of impact on model parameter updates. Thus, we challenge PoisonSpot by decreasing the perturbation budget ϵ for the trigger to decrease the strength of the attack. For Label-Consistent attack, the previous results were obtained using an $\epsilon = \frac{8}{255}$ for the PGD adversarial perturbation. We decreased it to $\frac{4}{255}$ and also changed the visibility of the trigger by decreasing the amplitude of the trigger to 32 pixels. This decreases the strength of the attack, but would potentially make it difficult for PoisonSpot to precisely filter poisonous samples. For the Narcissus attack, the perturbation budget for the ℓ_∞-norm ball was $\frac{16}{255}$. We first reduce it to $\frac{8}{255}$ and then to $\frac{6}{255}$.

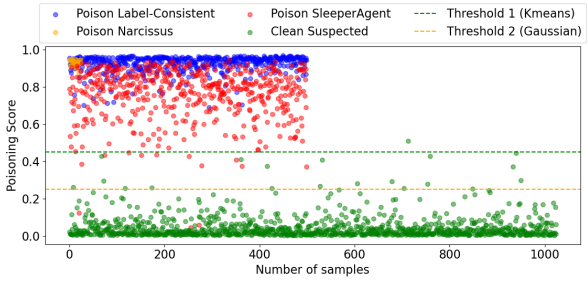
The results of PoisonSpot against the decreased perturbation budgets for both attacks are shown in Table 9. The upstream evaluation for Narcissus is different compared to when the strength of the attack was high in Table 5. It is not a perfect separation now because the poisoned samples are not as strong as before (hence, their effect on the model parameter updates is not huge). This results in low TPR and high FPR as there is some overlap in the distribution of clean and poisonous samples. However, when we examine the results of the downstream evaluation, which is the ultimate indicator of the robustness of any defense, PoisonSpot performs very well because it removes the most poisonous samples from the training set and limits the downstream **ASR** to less than 10%. The remaining ones have minimal effect on the poisonous model parameter updates (hence, would not hugely increase the ASR). One cost we would pay is a slight increase in FPR, which would result in a lowering of ACC, but we can see that there is a negligible decrease in ACC. For Label-Consistent, the results are nearly perfect, suggesting that the backdoor trigger for Label-Consistent attack is easier to detect than the noisy trigger used in Narcissus. ***Overall, we conclude that PoisonSpot is robust to changes in perturbation strengths of backdoor poisoning attacks***.

Mixed Triggers Attack: When an adversary employs multiple attacks simultaneously, poison samples from different types of attacks update parameters differently. Clean model parameter updates should still be different from poisonous model parameter updates, but the poisonous model parameter updates may not have common relevant parameter dimensions for diverse attacks, which may make the model less confident in detecting poisonous samples. To simulate such an adaptive attack, we craft backdoor trigger-carrying samples using multiple attack strategies against a target class. The results for two attacks (Label-Consistent and Narcissus) are shown in Table 10. The results for three attacks (Label-Consistent, Narcissus, and Sleeper-Agent) are shown in Table 11. For all attacks, we used the poison ratio that offered high ASR. For the two mixed attacks, PoisonSpot perfectly identified the poisonous samples for both attacks using the right threshold choice. However, we can see from Figure 4 **a)** that the distribution for Label-Consistent and Narcissus attacks is different. The model is less confident for Narcissus samples compared to Label-Consistent poisonous samples. Nonetheless, the clean distribution is still separate from both poisonous distributions. When using a mixture of three attacks, PoisonSpot perfectly identified Label-Consistent and Narcissus poisonous samples and identified the majority of Sleeper-Agent samples. We can see from Figure 4 **b)** that there are three distributions for each attack with very minimal overlap. The model is the least confident about Sleeper-Agent poisonous samples, where some of Sleeper-Agent samples overlap with the clean samples. This was also the case when we evaluated Sleeper-Agent individually (Table 7). However, the downstream evaluation suggests low ASR even if we left some of the poisoned samples from Sleeper-Agent undetected. This is also the case here, as we can see from the downstream evaluation results in Table 11, where we have low ASR for all attacks. ***Overall, we conclude that PoisonSpot is robust against a mixture of attacks that occur on the same target class simultaneously.***

Increased Poisoning Percentage Attack: We increase the poisoning percentage to test PoisonSpot. To simulate this attack, we varied $\mathbf{PR}_{\mathcal{D}c \cup \mathcal{D}u}$ up to 10%. The results are shown in Table 12 and 13. From the results, we can see that PoisonSpot is robust to changes in poison ratio even when 100% of the target class is poisoned. Even though, based on our threat model, we should have a high percentage of $\mathcal{D}_c$ as compared to $\mathcal{D}_u$, PoisonSpot could theoretically be extended to the case where $\mathcal{D}_c$ is very small and our entire training set consists of $\mathcal{D}_u$. This is supported by the result that PoisonSpot perfectly detected the poisonous samples for 10% $\mathbf{PR}(\mathcal{D}_c \cup \mathcal{D}_u)$ where all the target class is poisoned. Since the approach for PoisonSpot is exclusive for each target class, if all of the training set was unknown or poisonous, PoisonSpot could detect it by tracking the provenance data for each target class.



(a) Two attacks: Label-Consistent+Narcissus.



(b) Three attacks: Label-Consistent+Narcissus+Sleeper-Agent.

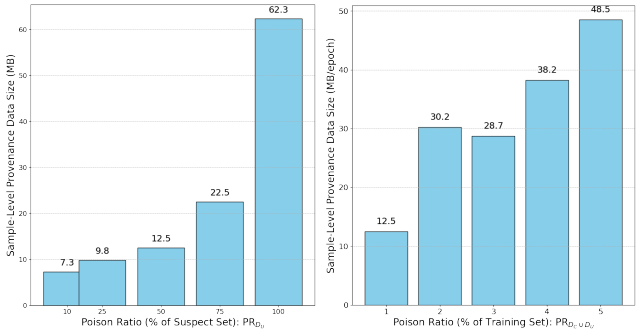
Figure 4: Resilience to mixed triggers attacks.

Attack	Label-Consistent [40]	Label-Consistent [40]	Narcissus [46]	Narcissus [46]
PR($\mathcal{D}_c \cup \mathcal{D}_u$) , PR($\mathcal{D}_u$) , TC	**4%, 50% Class 2**	**5%, 50% Class 2**	**4%, 50% Class 2**	**5%, 50%, Class 2**
Original ACC	95.16	95.10	95.03	94.66
Original ASR	99.94	100.0	98.04	98.76
TPR	100	100	100	100
FPR	0.05	0	0.15	0
ACC	94.75 (↓0.31)	94.92 (↓0.18)	95.11 (↑0.08)	94.42 (↓0.24)

Table 12: Resilience to poisoning percentage manipulation.

Attack	Label-Consistent [40]	Label-Consistent [40]	Narcissus [46]	Narcissus [46]
PR($\mathcal{D}_c \cup \mathcal{D}_u$) , PR($\mathcal{D}_u$) , TC	**7.5%, 75% Class 2**	**10%, 100%, Class 2**	**7.5%, Class 2**	**10%, 100%, Class 2**
Original ACC	94.38	86.25	93.77	86.16
Original ASR	99.98	100	100	100
TPR	100	100	100	100
FPR	0.16	0	0.08	0
ACC	94.26 (↓0.12)	86.43 (↑0.18)	94.52 (↑0.75)	86.01 (↓0.15)

Table 13: Resilience to poisoning percentage manipulation.



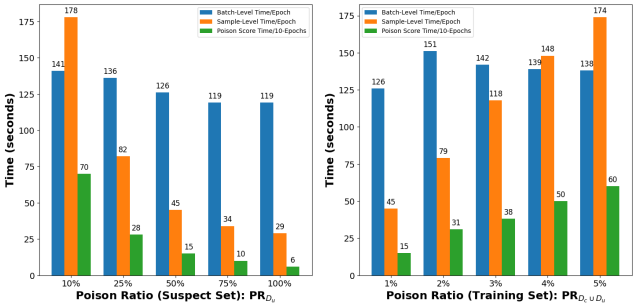
(a) Suspect Set (b) Training Set

Figure 5: Provenance memory footprint vs. poison ratios.

7.6 RQ5: Overhead Analysis

We analyze the computational overhead and memory footprint for PoisonSpot based on CIFAR-10. Our hardware setup for all numbers reported is an NVIDIA GeForce RTX 4090 GPU with 24 GB of GPU memory and 251 GB of CPU memory.

Figure 6 shows the computational overhead of PoisonSpot, which varies across different poison ratios based on the training set: **PR**($\mathcal{D}_c \cup \mathcal{D}_u$), shown in Figure 6b and the unknown/suspect set: **PR**($\mathcal{D}_u$), shown in Figure 6a. The maximum time required occurs at a 1% **PR**($\mathcal{D}_c \cup \mathcal{D}_u$) and a 10% **PR**($\mathcal{D}_u$), where the entire target set is



(a) Suspect Set (b) Training Set

Figure 6: Computational overhead vs. poison ratios.

suspected. In this instance, it takes a total of 319 seconds to capture provenance for both batch-level and sample-level tracking. The batch-level tracking is performed over 10 epochs, while the sample-level tracking spans 25 epochs, which takes less than 6000 seconds in total. For the results presented earlier, most of the results have been evaluated for 10 epochs for both batch-level and sample-level while a few of them with 10% **PR**($\mathcal{D}_u$) have been evaluated for 25 epochs. Thus, we can say the maximum number of epochs needed to capture provenance for PoisonSpot is at most 25 epochs. Figure 5 shows per epoch memory footprint of PoisonSpot. Sample-level provenance tracking is a component of PoisonSpot that requires a significant amount of memory. The highest memory usage is observed when the entire target set is considered suspected (1% poison ratio for the **PR**($\mathcal{D}_c \cup \mathcal{D}_u$) and 50% poison ratio for the **PR**($\mathcal{D}_u$)). In this scenario, it consumed 62.3 MB per epoch for 25 epochs, resulting in a total of 1555 MB. ***Given the periodic retraining scenario, the computational overhead and memory footprint are practically acceptable for a model trainer.***

8 Conclusion

This paper presents PoisonSpot, a practical system that leverages fine-grained training provenance tracking for precise detection of clean-label backdoor attacks. Throughout a training session, PoisonSpot effectively monitors and analyzes the impact of suspect samples on model parameter updates, allowing it to distinguish between clean and poisonous data with high precision. PoisonSpot

is robust across various datasets, attack types, and poisoning ratios, outperforming the state-of-the-art defense. Furthermore, PoisonSpot maintains strong resilience against adaptive attacks. PoisonSpot is the first system to tailor dynamic taint tracking principles for the precise detection of clean-label backdoor poisoning attacks.

Acknowledgments

We thank the anonymous reviewers for their insightful feedback that improved the paper. This work was supported by the National Science Foundation (NSF) award CNS-2238084.

References

[1] Aghakhani, H., Meng, D., Wang, Y., Kruegel, C., and Vigna, G. Bullseye polytope: A scalable clean-label poisoning attack with improved transferability. In *IEEE European Symposium on Security and Privacy, EuroS&P 2021* (2021), IEEE, pp. 159–178.

[2] Amini, M.-R., Feofanov, V., Pauletto, L., Hadjadj, L., Devijver, E., and Maximov, Y. Self-training: A survey. *Neurocomputing 616* (2025), 128904.

[3] Baracaldo, N., Chen, B., Ludwig, H., and Safavi, J. A. Mitigating poisoning attacks on machine learning models: A data provenance based approach. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security, AISec@CCS 2017* (2017), ACM, pp. 103–110.

[4] Baracaldo, N., Chen, B., Ludwig, H., Safavi, J. A., and Zhang, R. Detecting poisoning attacks on machine learning in iot environments. In *2018 IEEE International Congress on Internet of Things, ICIOT 2018* (2018), IEEE Computer Society, pp. 57–64.

[5] Biggio, B., Nelson, B., and Laskov, P. Poisoning attacks against support vector machines. In *Proceedings of the 29th International Conference on Machine Learning, ICML 2012* (2012), icml.cc / Omnipress.

[6] Chan, A., and Ong, Y.-S. Poison as a cure: Detecting & neutralizing variable-sized backdoor attacks in deep neural networks, 2019.

[7] Chen, B., Carvalho, W., Baracaldo, N., Ludwig, H., Edwards, B., Lee, T., Molloy, I. M., and Srivastava, B. Detecting backdoor attacks on deep neural networks by activation clustering. In *Workshop on Artificial Intelligence Safety 2019 co-located with the Thirty-Third AAAI Conference on Artificial Intelligence 2019 (AAAI-19), Honolulu* (2019), vol. 2301 of *CEUR Workshop Proceedings*, CEUR-WS.org.

[8] Chen, X., Liu, C., Li, B., Lu, K., and Song, D. Targeted backdoor attacks on deep learning systems using data poisoning. *CoRR abs/1712.05526* (2017).

[9] Chin, E., and Wagner, D. Efficient character-level taint tracking for java. In *Proceedings of the 2009 ACM workshop on Secure web services* (2009), pp. 3–12.

[10] Cinà, A. E., Grosse, K., Demontis, A., Vascon, S., Zellinger, W., Moser, B. A., Oprea, A., Biggio, B., Pelillo, M., and Roli, F. Wild patterns reloaded: A survey of machine learning security against training data poisoning. *ACM Comput. Surv. 55*, 13s (2023), 294:1–294:39.

[11] Coates, A., Ng, A., and Lee, H. An analysis of single-layer networks in unsupervised feature learning. *Journal of Machine Learning Research - Proceedings Track 15* (01 2011), 215–223.

[12] Deng, J., Dong, W., Socher, R., Li, L., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2009)* (2009), IEEE Computer Society, pp. 248–255.

[13] Gao, Y., Xu, C., Wang, D., Chen, S., Ranasinghe, D. C., and Nepal, S. Strip: A defence against trojan attacks on deep neural networks, 2020.

[14] Gaspari, F. D., Hitaj, D., and Mancini, L. V. Have you poisoned my data? defending neural networks against data poisoning. In *Computer Security - ESORICS 2024* (2024), vol. 14982 of *Lecture Notes in Computer Science*, Springer, pp. 85–104.

[15] Geiping, J., Fowl, L., Huang, W. R., Czaja, W., Taylor, G., Moeller, M., and Goldstein, T. Witches' brew: Industrial scale data poisoning via gradient matching, 2021.

[16] Gu, T., Liu, K., Dolan-Gavitt, B., and Garg, S. Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access 7* (2019), 47230–47244.

[17] Hastie, T., Tibshirani, R., Friedman, J., Hastie, T., Tibshirani, R., and Friedman, J. Random forests. *The elements of statistical learning: Data mining, inference, and prediction* (2009), 587–604.

[18] Hayase, J., Kong, W., Somani, R., and Oh, S. Spectre: defending against backdoor attacks using robust statistics. In *Proceedings of the 38th International Conference on Machine Learning* (18–24 Jul 2021), vol. 139 of *Proceedings of Machine Learning Research*, PMLR, pp. 4129–4139.

[19] Jackson, J.-N. Exploring training provenance for clues of data poisoning in machine learning. Master's thesis, University of Michigan, Dearborn, MI, United States, 2023.

[20] Jia, J., Liu, Y., Cao, X., and Gong, N. Z. Certified robustness of nearest neighbors against data poisoning and backdoor attacks, 2021.

[21] Krizhevsky, A., Nair, V., and Hinton, G. Cifar-10 (canadian institute for advanced research).

[22] Li, Y., Li, Y., Wu, B., Li, L., He, R., and Lyu, S. Invisible backdoor attack with sample-specific triggers. In *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021* (2021), IEEE, pp. 16443–16452.

[23] Li, Y., Lyu, X., Koren, N., Lyu, L., Li, B., and Ma, X. Anti-backdoor learning: Training clean models on poisoned data, 2021.

[24] Liu, Y., Ma, S., Aafer, Y., Lee, W., Zhai, J., Wang, W., and Zhang, X. Trojaning attack on neural networks. In *25th Annual Network and Distributed System Security Symposium, NDSS 2018* (2018), The Internet Society.

[25] Liu, Y., Ma, X., Bailey, J., and Lu, F. Reflection backdoor: A natural backdoor attack on deep neural networks. In *Computer Vision – ECCV 2020: 16th European Conference* (2020), Springer, pp. 182–199.

[26] Loshchilov, I., and Hutter, F. SGDR: stochastic gradient descent with warm restarts. In *5th International Conference on Learning Representations, ICLR 2017, Conference Track Proceedings* (2017), OpenReview.net.

[27] Ma, W., Wang, D., Sun, R., Xue, M., Wen, S., and Xiang, Y. The "beatrix" resurrections: Robust backdoor detection via gram matrices. In *30th Annual Network and Distributed System Security Symposium, NDSS 2023* (2023), The Internet Society.

[28] Nelson, B., Barreno, M., Chi, F. J., Joseph, A. D., Rubinstein, B. I. P., Saini, U., Sutton, C., Tygar, J. D., and Xia, K. Exploiting machine learning to subvert your spam filter. In *First USENIX Workshop on Large-Scale Exploits and Emergent Threats, LEET '08* (2008), USENIX Association.

[29] Nguyen, T. A., and Tran, A. Input-aware dynamic backdoor attack. *Advances in Neural Information Processing Systems 33* (2020), 3454–3464.

[30] Nguyen, T. A., and Tran, A. T. Wanet - imperceptible warping-based backdoor attack. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria* (2021), OpenReview.net.

[31] Pan, M., Zeng, Y., Lyu, L., Lin, X., and Jia, R. ASSET: robust backdoor data detection across a multiplicity of deep learning paradigms. In *32nd USENIX Security Symposium, USENIX Security 2023* (2023), USENIX Association, pp. 2725–2742.

[32] Papagiannis, I., Migliavacca, M., and Pietzuch, P. PHP aspis: Using partial taint tracking to protect against injection attacks. In *2nd USENIX Conference on Web Application Development (WebApps 11)* (2011).

[33] Qi, X., Xie, T., Wang, J. T., Wu, T., Mahloujifar, S., and Mittal, P. Towards A proactive ML approach for detecting backdoor poison samples. In *32nd USENIX Security Symposium, USENIX Security 2023* (2023), USENIX Association, pp. 1685–1702.

[34] Saha, A., Subramanya, A., and Pirsiavash, H. Hidden trigger backdoor attacks. In *AAAI* (2020), AAAI Press, pp. 11957–11965.

[35] Severi, G., Meyer, J., Coull, S. E., and Oprea, A. Explanation-guided backdoor poisoning attacks against malware classifiers. In *30th USENIX Security Symposium, USENIX Security 2021* (2021), USENIX Association, pp. 1487–1504.

[36] Sharif, M., Bhagavatula, S., Bauer, L., and Reiter, M. K. Accessorize to a crime: Real and stealthy attacks on state-of-the-art face recognition. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016* (2016), ACM, pp. 1528–1540.

[37] Souri, H., Fowl, L., Chellappa, R., Goldblum, M., and Goldstein, T. Sleeper agent: Scalable hidden trigger backdoors for neural networks trained from scratch. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022* (2022).

[38] Stokes, J. W., England, P., and Kane, K. Preventing machine learning poisoning attacks using authentication and provenance. In *2021 IEEE Military Communications Conference, MILCOM 2021* (2021), IEEE, pp. 181–188.

[39] Tang, D., Wang, X., Tang, H., and Zhang, K. Demon in the variant: Statistical analysis of DNNs for robust backdoor contamination detection. In *30th USENIX Security Symposium (USENIX Security 21)* (Aug. 2021), USENIX Association, pp. 1541–1558.

[40] Turner, A., Tsipras, D., and Madry, A. Label-consistent backdoor attacks. *arXiv preprint arXiv:1912.02771* (2019).

[41] Wang, B., Yao, Y., Shan, S., Li, H., Viswanath, B., Zheng, H., and Zhao, B. Y. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *2019 IEEE Symposium on Security and Privacy, SP 2019* (2019), IEEE, pp. 707–723.

[42] Wenger, E., Passananti, J., Bhagoji, A. N., Yao, Y., Zheng, H., and Zhao, B. Y. Backdoor attacks against deep learning systems in the physical world. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual* (2021), Computer Vision Foundation / IEEE, pp. 6206–6215.

[43] Wu, D., and Wang, Y. Adversarial neuron pruning purifies backdoored deep models. *Advances in Neural Information Processing Systems 34* (2021), 16913–16925.

[44] Yang, Y., Liu, T. Y., and Mirzasoleiman, B. Not all poisons are created equal: Robust training against data poisoning. In *International Conference on Machine Learning, ICML 2022* (2022), vol. 162 of *Proceedings of Machine Learning Research*, PMLR, pp. 25154–25165.

k	$\mathcal{F}_{\text{impt1}}$	$\mathcal{F}_{\text{impt2}}$	TPR (%)	FPR (%)
0.5	12,350	991	94.16	0.00
0.875	10,575	979	94.16	0.00
1.0	10,152	969	94.20	0.00
1.25	9,456	938	94.00	0.00
1.625	8,688	936	94.08	0.00
2.0	8,020	912	94.08	0.00

Table 14: Threshold analysis over first-phase feature selection process.

τ_2	$\mathcal{F}_{\text{impt2}}$	TPR (%)	FPR (%)
0.00001	4,285	94.20	0.00
0.00012	832	94.24	0.00
0.00023	555	93.52	0.00
0.00034	419	93.16	0.00
0.00045	354	93.52	0.00
0.00056	304	93.16	0.00
0.00067	235	92.64	0.00
0.00078	211	92.92	0.00
0.00089	186	91.48	0.00
0.00100	172	91.48	0.00

Table 15: Threshold analysis over the second-phase feature selection process.

[45] Yao, Y., Li, H., Zheng, H., and Zhao, B. Y. Latent backdoor attacks on deep neural networks. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019* (2019), ACM, pp. 2041–2055.

[46] Zeng, Y., Pan, M., Just, H. A., Lyu, L., Qiu, M., and Jia, R. Narcissus: A practical clean-label backdoor attack with limited information. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023* (2023), ACM, pp. 771–785.

[47] Zeng, Y., Park, W., Mao, Z. M., and Jia, R. Rethinking the backdoor attacks' triggers: A frequency perspective, 2022.

[48] Zhong, N., Qian, Z., and Zhang, X. Imperceptible backdoor attack: From input space to feature representation. *arXiv preprint arXiv:2205.03190* (2022).

Appendix

8.1 Threshold Analysis

The sensitive thresholds for PoisonSpot are the two feature selection parameters: k and τ_2 which control the number of $\mathcal{F}_{impt1}$ and $\mathcal{F}_{impt2}$, respectively. The first threshold, k, controls which features have the largest difference between poisoned and clean parameter updates by selecting those whose values lie k number of standard deviations from the overall mean.

Table 14 shows how varying k affects PoisonSpot's performance. The results indicate minimal variation in the final outcome. This is because the first phase threshold simply reduces the massive model parameter feature space, while the more critical second phase selection further refines this set which yields a similar range of retained features regardless of k. We use $k = 1$ by default.

The second threshold, τ_2, sets the Random Forest feature importance cut-off value required to further refine the important features. Table 15 shows that lower values of τ_2 capture more important features and improve performance. In PoisonSpot, we use $\tau_2 = 0.0001$ as the default value. For both analyses, we used Sleeper-Agent with $\text{PR}(D_c \cup D_u) = 1\%$ and $\text{PR}(D_u) = 10\%$.

8.2 Results on Additional Datasets

We further validate PoisonSpot on two additional datasets: ImageNet-100 [12] and STL-10 [11] dataset. These results are shown in Table 16. The STL-10 dataset is evaluated for Sleeper-Agent and Hidden-Trigger Backdoor while the ImageNet-100 dataset is evaluated for Hidden-Trigger. For STL-10 dataset, we resized images to 224x224 to fit our ViT model which was trained on ImageNet-1K. This dataset shows how our model performs when the image dimension increases and also when the model gets complex as the ViT model has much more parameters compared to ResNet18. The upstream evaluation was perfect for Sleeper Agent and almost close to perfection on Hidden-Trigger which makes the downstream evaluation very ideal. The poison ratios used are similar to CIFAR-10 for both datasets. For ImageNet-100, the upstream evaluation was not perfect, with high FPR for Hidden-Trigger. However, the downstream evaluation was satisfactory as the downstream ACC did not drop for Hidden-Trigger. The evaluation also validates our intuition that PoisonSpot tracking suspect samples and filtering them out based on their model parameter update provenance is overall effective even if some poison samples are overlooked.

Attack	**Sleeper-Agent**		**Hidden-Trigger**		**Avg.**	**Worst**
PR($\mathcal{D}_c \cup \mathcal{D}_u$) , PR($\mathcal{D}_u$) , TC , SC	**5%, 50%, Class 1, Class 0**		**5%, 50%, Class 4, Class 3**			
STL-10						
Upstream Evaluation						
	TPR	**FPR**	**TPR**	**FPR**	**Avg. TPR**	**Worst FPR**
ASSET	90.8	0.34	99.2	0.19	95.0	0.34
PoisonSpot	100	0	99.6	0.4	99.8	0.4
Downstream Evaluation						
	ASR	**ACC**	**ASR**	**ACC**	**Avg. ASR**	**Worst ACC**
No Defense (ours)	100	96.80	73.62	94.83	86.81	94.83
No Defense (reported [31])	93.6	98.6	68.1	98.5	66.12	98.5
ASSET (reported [31])	15.4	98.1 (↓ 0.5)	0.36	98.5 (·)	7.88	98.1
PoisonSpot	0.0	96.40 (↓ 0.4)	0.38	95.76 (↑ 0.45)	0.19	95.76
ImageNet						
Attack			**Hidden-Trigger**		**Avg.**	**Worst**
PR($\mathcal{D}_c \cup \mathcal{D}_u$) , PR($\mathcal{D}_u$) , TC , SC			**5%, 50%, Class 40, Class 30**			
Upstream Evaluation						
			TPR	**FPR**	**Avg. TPR**	**Worst FPR**
PoisonSpot			98.8	5.2	98.8	5.2
Downstream Evaluation						
			ASR	**ACC**	**Avg. ASR**	**Worst ACC**
No Defense (ours)			96.00	75.08	93.91	75.08
PoisonSpot			2.00	73.60 (↑ 0.18)	1.06	73.60

Table 16: Additional datasets comparison of PoisonSpot with ASSET under **RQ2** and **RQ3** on ImageNet and STL-10 using upstream and downstream evaluations.

8.3 Datasets, Models, and Hyper-Parameters

ImageNet-100 is a subset of the Tiny-ImageNet-200 dataset, consisting of 100 randomly selected classes, with $50K$ images. Each image is resized to 224×224 pixels to fit our ViT model, which was pre-trained on ImageNet-1K. The CIFAR-10 [21] dataset consists of $60K$, 32×32 color images, equally distributed across 10 classes, with $50K$ training images and $10K$ test images. The STL-10 dataset comprises 10 classes, with the original image dimensions of 96×96 resized to 224×224 similar to ImageNet-100 to fit our ViT model. For all experiments, we apply combinations of data augmentation techniques to each sample, such as normalization, random horizontal flipping, and random cropping. For CIFAR-10 and ImageNet, we train a ResNet18 model, and a ViT model for

ImageNet-100 and STL-10. For CIFAR-10, we use the SGD optimizer with the learning rate starting from 0.1, updated using a CosineAnnealingLR scheduler [26]. For STL-10 and ImageNet-100, we use the Adam optimizer with a learning rate of 0.0001. The typical batch size is 512 for CIFAR-10, and 64 for both STL-10 and ImageNet. The batch size used for PoisonSpot varies according to the percentage of samples in $\mathcal{D}_u$ compared to our target training set. We use 200 epochs for training from scratch and 50 epochs for fine-tuning.